

Towards a Flexible Python Framework for Developing Domain-Specific Languages

by

Shreya Chaudhary

S.B. Computer Science and Engineering, MIT 2026

Submitted to the Department of Electrical Engineering and Computer Science
in partial fulfillment of the requirements for the degree of

MASTER OF ENGINEERING IN ELECTRICAL ENGINEERING AND COMPUTER
SCIENCE

at the

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

May 2026

© 2026 Shreya Chaudhary. All rights reserved.

The author hereby grants to MIT a nonexclusive, worldwide, irrevocable, royalty-free license to exercise any and all rights under copyright, including to reproduce, preserve, distribute and publicly display copies of the thesis, or release the thesis under an open-access license.

Authored by:	Shreya Chaudhary Department of Electrical Engineering and Computer Science May 15, 2026
Certified by:	Saman Amarasinghe Professor of Computer Science and Engineering, Thesis Supervisor
Accepted by:	Katrina LaCurts Chair, Master of Engineering Thesis Committee

Towards a Flexible Python Framework for Developing Domain-Specific Languages

by

Shreya Chaudhary

Submitted to the Department of Electrical Engineering and Computer Science
on May 15, 2026 in partial fulfillment of the requirements for the degree of

MASTER OF ENGINEERING IN ELECTRICAL ENGINEERING AND COMPUTER
SCIENCE

ABSTRACT

Domain Specific Languages (DSLs) are programming languages optimized for specific domains. However, the development of DSLs often requires writing redundant code, ranging from intermediate representation (IR) data structures like abstract syntax trees (ASTs) and control flow graphs (CFGs) to operations across them such as data flow analyses. Some frameworks for developing DSLs exist to increase the amount of reuse, but they are often difficult to understand or have strong limitations on the intermediate representation.

Compilers have historically been built with low-level languages such as C++ using complex frameworks such as LLVM and MLIR. However, high-level languages like Python have increased in popularity. Recent tools such as Codon can compile Python to have comparable performance to C++, allowing for the development of Python frameworks without the performance costs.

Consequently, this thesis explores developing a flexible Python framework for creating DSLs. The framework developed provides reusable functions that can operate on intermediate representations to provide optimizations out-of-box. In addition, this framework is partially with Codon to reduce the overhead of using Python, resulting in comparable performance. We conclude with an implementation of finch-tensor-lite in this platform with a comparison of runtime.

Thesis supervisor: Saman Amarasinghe

Title: Professor of Computer Science and Engineering

Contents

Title page	1
Abstract	3
List of Figures	9
List of Tables	11
1 Introduction	15
2 Related Works	18
2.1 Reuse across Compilers	18
2.2 Python DSLs	20
2.2.1 Python Tools for DSLs	20
2.2.2 Python DSLs	20
3 System Overview	22
4 Python ADTs	26
4.1 Chapter Summary	26
4.2 Abstract Data Types (ADT) Motivation and Design Goals	27
4.3 Python Limitations	29
4.3.1 Python Type Checking Support	29
4.4 mypy Background	29
4.4.1 Semantic Analyzer Pass Details	30
4.4.2 defer	31
4.4.3 mypy Types	31
4.4.4 mypy Hooks	32
4.5 Python Abstract Data Types	32
4.5.1 ADT Potential Design Choices	32
4.5.2 ADT Syntax	34
4.5.3 ADT Plugin Implementation Details	35
4.6 Reuse	36
4.6.1 Type Variables	39
4.6.2 Reuse Nodes within ADT	40
4.7 Additional Utilities	42

4.7.1	Dataclass Type Checking	42
4.7.2	Dataclass Hashing	43
5	Functors	44
5.1	Chapter Summary	44
5.2	Motivation	44
5.3	Functors	45
5.3.1	Our Implementation	46
5.4	CFG Construction Algorithm	51
5.4.1	Algorithmic Overview	51
5.4.2	Functor Representation	52
5.5	CFG to SSA CFG	54
5.5.1	Dominators	54
5.5.2	Phi Node Insertion	55
5.5.3	Variable Renaming	56
5.5.4	SSA Example	57
5.5.5	SSA CFG Functor	58
5.6	Copy Propagation (with SSA)	60
5.6.1	Copy Propagation Definition	60
5.6.2	SSA Copy Propagation	61
5.6.3	SSA Copy Propagation Functor	61
5.7	Copy Propagation (without SSA)	61
5.7.1	Algorithm Overview	62
5.7.2	Functor Overview	62
6	Codon Compatibility	64
6.1	Chapter Summary	64
6.2	Codon Motivation and Background	64
6.2.1	Speedup Expectations	65
6.3	Codon Integration End Result	66
6.4	Codon Compilation Of Our Compiler Infrastructure	69
6.4.1	Accessing Attributes	69
6.4.2	isinstance and Any type	73
6.4.3	Miscellaneous Changes	74
6.5	Alternative Integrations with Codon	75
6.5.1	PyExt	75
6.5.2	Codon JIT	77
7	Evaluation	79
7.1	Evaluation Setup	79
7.1.1	Evaluation Metric	79
7.1.2	Validating Correctness	80
7.2	Results and Discussion	80
7.2.1	Case Study: Dot Product	80
7.2.2	Discussion of Results	83

8	Future Work	84
8.1	Recommendations for Codon	84
8.2	Extra Features	85
9	Conclusion	86
	References	87

List of Figures

3.1	Overview of the two main components in the software created and their features. The IR construction provides custom types and reuse on the IR level. The dataflow analyses provide reusable functors for compiler algorithms and building CFGs and is partially compiled with Codon for speed	23
3.2	Overview of ways to plug into our software to build a compiler. Continuation from Figure 3.1. A summary of how the compiler exploration can be used in different stages of compiler development. The green box indicates features offered by our software and the arrows indicate any combination that the user may use. The Python compiler package provides flexibility, allowing users to use our custom ADTs and/or functors and/or use their own implementations. At any stage, a user can plug into our software.	24
4.1	Breakdown of how an ADT invokes the mypy hooks to create Unions. See how it's used to ensure completeness in Figure 4.2	37
4.2	Continuation from Figure 4.1. Breakdown of how the ADT previously defined can be used for completeness and recursion	38
5.1	Example of common statements that change dataflow	51
5.2	Example of all of the stages of SSA on an if/else statement	58
5.3	Example of copy propagation on a non-SSA CFG. The yellow highlights indicate local copy propagation and the green highlight indicates global copy propagation.	63
7.1	CFG generated from the dot product code	81

List of Tables

1.1	Lines of Code in DSL Repositories	16
1.2	Comparison of how different compiler frameworks gain reuse and automation. Our framework is closer to MLIR and has some components from staging . .	17
3.1	Overview of the progress of each components	25
4.1	The three main mypy passes. The pass of most interest for developing Python ADTs is the semantic analyzer.	30
4.2	mypy Types	32
4.3	mypy hooks with additional information on whether the hook returns a type that can be modified and whether they were used in the final ADT extension implementation	33
5.1	Comparison of StandardML’s functor implementation to our Python imple- mentation	45
5.2	Functors developed and discussed. Note that the functors that mutate the input return None and instead perform the tranformation on the input. . . .	46
5.3	The mapping used to identify the corresponding instruction and/or statement based on input functors when building the CFG	52
6.1	Codon Compilation Issues and Successes Summary	65
7.1	Runtime of CFG Creation	80

Listings

4.1	Outline of requirements for ADT	27
4.2	Example of a recursive ADT element: a for loop!	27
4.3	Example of assert never	28
4.4	mypy execution command	29
4.5	Current syntax for ADT. The ADT Super has members Temp_ADT.A and Temp_ADT.B	34
4.6	CExpr without Type Variables	39
4.7	CExpr with Type Variables	40
4.8	CExpr setup example	41
4.9	CppExpr with reuse from CExpr	41
4.10	Example of importing custom dataclass	42
5.1	Signature for a Print CFG Functor	46
5.2	Print CFG Functor with helper functions to iterate through the basic blocks and instructions	47
5.3	Full functor for printing a CFG	47
5.4	Simple implementation using the print CFG Functor	48
5.5	Simple implementation using linked lists for the print CFG Functor	49
5.6	Get Condition Helper Function	52
5.7	Type Signature of the build CFG function	53
5.8	Dominance Frontier Algorithm	55
5.9	Phi Node Insertion Algorithm	55
5.10	Variable Renaming Algorithm	56
5.11	Update Reaching Definition Algorithm	57
5.12	Simple If/Else for Demonstration CFG	57
5.13	Signature of SSA algorithm	59
5.14	Sample Code Pre-Copy Propagation	60
5.15	Sample Code Post-Copy Propagation	60
5.16	SSA Copy Propagation Functor Signature	61
5.17	Copy Propagation Functor Signature	62
6.1	Functor in Python	66
6.2	Functor in Codon	67
6.3	ADT in Python	67
6.4	ADT in Codon	68
6.5	Subclassing that fails in Codon	70
6.6	NoneType Error	70

6.7	Codon Annotation with Optional	71
6.8	Subclassing that works in Codon	72
6.9	Casting to Optional Type	74
6.10	Subclassing fails to compile with Codon Pyext	75
6.11	Example AssemblyExpression	76
6.12	Creating an AssemblyExpression	76
6.13	Codon JIT Example	77
6.14	Codon JIT Example for a Function	77
7.1	Finch Code for dot product	80
7.2	Recreated Finch Code for dot product	82

Chapter 1

Introduction

Domain Specific Languages (DSLs) are programming languages specialized for a specific domain, catering to needs in this domain such as performance, specialized capabilities, or literate programming. They contrast general purpose languages like C or Python which aims to provide support for a general range of operations without catering to specific domains and needs.

DSLs are prevalent in both academia and industry and have been created for a wide range of domains. Popular DSLs include Halide for image processing [1], Finch for sparse tensor programming [2], StreamIt for streaming applications [3], GraphIt for graph applications [4], taco for tensor algebra kernel compilation [5], SQL for database queries [6], Verilog for FPGA programming [7], and Bluespec for digital electronic hardware design [8].

However, the development of DSLs can often be repetitive and cumbersome. Table 1.1 shows the lines of code for some DSLs. While the core purpose of a DSL is to add optimizations specific to a domain and workflow, DSLs still require considerable machinery to be functional. For example, all DSLs require some internal data structures including ASTs and CFGs and will perform operations over these IRs such as common subexpression elimination, value numbering, rewriting, and other queries. It's inefficient for DSL developers to re-implement these functionalities for every DSL they script. This limitation reveals an

opportunity for reuse across DSLs.

DSL	Lines of Code
Finch.jl	69k
Halide	350k
StreamIt	825k
GraphIt	89k

Table 1.1: Lines of Code in DSL Repositories

There has been efforts to streamline the process of developing DSLs, such as dialects through MLIR [9] and staging-based methods like in Scala [10], BuildIt [11], and Julia [12]. MLIR has a steep learning curve and can be unnecessarily complex. On the other hand, because staging must follow language semantics, staging-based methods require developer to "fight" the programming language to allow for their optimizations. The contrasts between these reveals a deeper problem in reuse: even simple optimizations rely on the definition and semantics of a language. These two approaches are at two extremes: staging relies on a host language to help with optimizations and in MLIR, users have to encode properties that allow for these optimizations (unless the optimizations only depend on SSA representation). This works for some optimizations but can get very complex.

Staging gets reuse by operating at an AST level and MLIR gets reuse by operating at the IR level. Our research aims to provide flexibility for developers to operate at both levels depending on what best suits their needs. We explore supporting both translating a language definition or AST and take advantage of an existing compiler and IR. In addition, it allows developers to create a custom IR which can still use some of our optimizations along with a user's custom optimizations.

This thesis is an exploration of developing compilers in Python. We aim to provide the ability to operate at both the IR and the AST level depending on user preference. We provide an exploratory prototype of a flexible and simple framework that provides users with reusable components when is most convenient for them. In addition, we explore using Codon help minimize the runtime overhead of Python compilers.

Compiler Framework	Reuse Level	Semantics	Automation
Staging	AST level	Host Language	Execution of Host Language
MLIR	IR level	User-Specified	Properties Encoded into SSA Infrastructure
Our Framework	Both IR and AST level	User-Specified	Properties Encoded into Infrastructure

Table 1.2: Comparison of how different compiler frameworks gain reuse and automation. Our framework is closer to MLIR and has some components from staging

In summary, our research is an exploration to providing a flexible, high-level interface to rapidly develop DSLs in Python with comparable performance to widely used compilers. Our prototype has three main design goals:

- **Flexible:** We don't constrain developers to a particular level of reuse or methods of creating IRs
- **Reasonable Performance:** Part of our pipeline has comparable performance to C++ compilers
- **Batteries Included:** We provide a library of IRs and functors which users can optionally use.

Chapter 2

Related Works

2.1 Reuse across Compilers

There is considerable redundancy between languages and language implementations. Differences between any two generally fall into three main categories: (1) modifications to the language semantics, (2) modifications to the syntax, the abstract syntax tree (AST), and (3) modifications to the backend or compiler itself independent though constrained by the semantics, including IRs, CFGs, and passes. Different methods to take advantage of reuse tends to focus on at least one of these categories.

One of the most popular frameworks for taking advantage of reuse is MLIR which provides reuse on the language level. MLIR is an open-source C++ compiler infrastructure framework and is commonly used in industry for compiler development. MLIR extends languages via dialects which are user-specified sets of types and operations across those types. These are then handled by MLIR under the hood to update a common SSA CFG intermediate representation that can have performance optimizations. However, this results in a rigid intermediate representation that may or may not provide corresponding benefits without sufficient work on the part of the compiler developer [9]

Another major idea for reuse is staging and meta programming. These ideas operate on

the AST level. For these applications, DSLs are embedded into some host language, allowing the program to take advantage of properties of the host language such as the type system. However, this comes with the drawback of creating workarounds to allow for compilation within the language itself: mainly the compilation is constrained by the semantics and implementation choices of the language and compiler. Some examples of embedded DSLs include eC++ [13] and BuiltIt [11] which are DSLs embedded into C++ and Delite which is embedded into Scala [14].

A compiler that falls in between these two ideas is PyTorch 2.0 [15] which provides two extensions to PyTorch: TorchDynamo and TorchInductor. TorchDynamo allows users to write normal Python which can be compiled via graph compilation. TorchInductor lowers some Python and PyTorch code into a much more constrained IR, which can be targeted Triton for GPUs and C++/OpenMP for CPUs.

Finally, LLVM, a predecessor and technically super project to MLIR, is similar to MLIR but provides reuse in the backend. LLVM is a source and target-independent optimizer with code generation support. It is one of the earlier approaches to compilers which is foundational to the aforementioned compilers. LLVM IR is an intermediate representation from a compiler’s output which can then undergo a series of optimizations. Similar to MLIR, this has a rigid intermediate representation but provides a variety of robust optimizations.

There are some compiler tools which don’t fall into any of these categories. One example of such tool is egraphs good (egg) [16]. Many compilers uses e-graphs to compactly represent and optimize programs.

Another approach that doesn’t fall into these categories is binder-unbound. This is a DSL for name binding that provides bind and unbound functionality. Creating an expression requires additional metadata of which variables are bounded. Using this information, the compiler can figure out properties such as alpha-equivalence, free variable calculation, and capture-avoiding substitution. This framework is general and can be implemented in any functional language. [17]

In summary, there are four main approaches to compilers: (1) AST-based staging, (2) CFGs with and without SSA which add to a predefined IR, (3) LLVM with the ability to abstract compiled code over hardware, and (4) hybrid and miscellaneous approaches such as egg.

2.2 Python DSLs

2.2.1 Python Tools for DSLs

Researchers have created Python-based DSLs. One major category of these DSLs are MLIR-adjacent DSLs which use similar architecture to MLIR. For example, xDSL provides a Pythonic interface with a text-based or Python IR which can generate MLIR [18]. In addition, there exists Python bindings for creating DSLs, such as llvmlite for LLVM Python bindings and Nelli [19] used for MLIR Python bindings.

Another type of DSL is Codon which is a Pythonic DSL for writing DSLs. Codon is a DSL for building high-performance DSLs in Python-like syntax. This employs a different type of reuse than MLIR: it tends to fall in between modifications to the language and modifications to the AST. [20] Other tools similar to Codon to make Python code faster include pypy and mojo.

2.2.2 Python DSLs

There is also previous work in creating DSLs in Python, and some notable DSLs include xDSL, FEniCS, various SQL ORMs, Firedrake, PyTorch, and Jax. xDSL is a framework for writing DSLs in Python that primarily targets SSA CFGs that are designed to mirror and interoperate with MLIR. FireDrake is a DSL for using finite element methods (FEMs). PyTorch, specifically TorchInductor, is a compiler in Python for writing machine learning code. Finally, Jax is a python DSL for numerical computation that uses MLIR as a backend

through a C++ interface.

Chapter 3

System Overview

We explored creating a framework for developing DSLs in Python. The experimental framework prototype is flexible and simple, includes parametric types, IRs, and functors, and provide users the option for both extensible data structures for IRs or built-in IRs for targets.

As summarized in Figure 3.1, our pipeline consists of two main components: (1) Python abstract data types (ADTs) and (2) functors to build control-flow graphs (CFGs) and perform dataflow analyses across these ADTs/CFGs. These abstractions in Python provide useful features for researchers to rapidly prototype compilers in Python. However, one major drawback of Python is its slowness due to it being an interpreted language. To help improve runtime, we used Codon to partially compile the functors which required removing the majority of our typing system.

Developing this framework firstly requires porting functionality commonly found in strongly typed programming languages to Python such as ADTs and type checking with mypy 4. There are also a number of nice-to-have features including hashing and type-checking for dataclasses 4. Next, we built a suite of functors that could be reused across different language definitions 5.

Finally, we built the IR for `finch-tensor-lite` to demonstration the reuse of the func-

tors and IRs 5. We closed with partially converting our functors to Codon and demonstrating a performance speedup from Codon relative to Finch 6.

Python Compiler Exploration

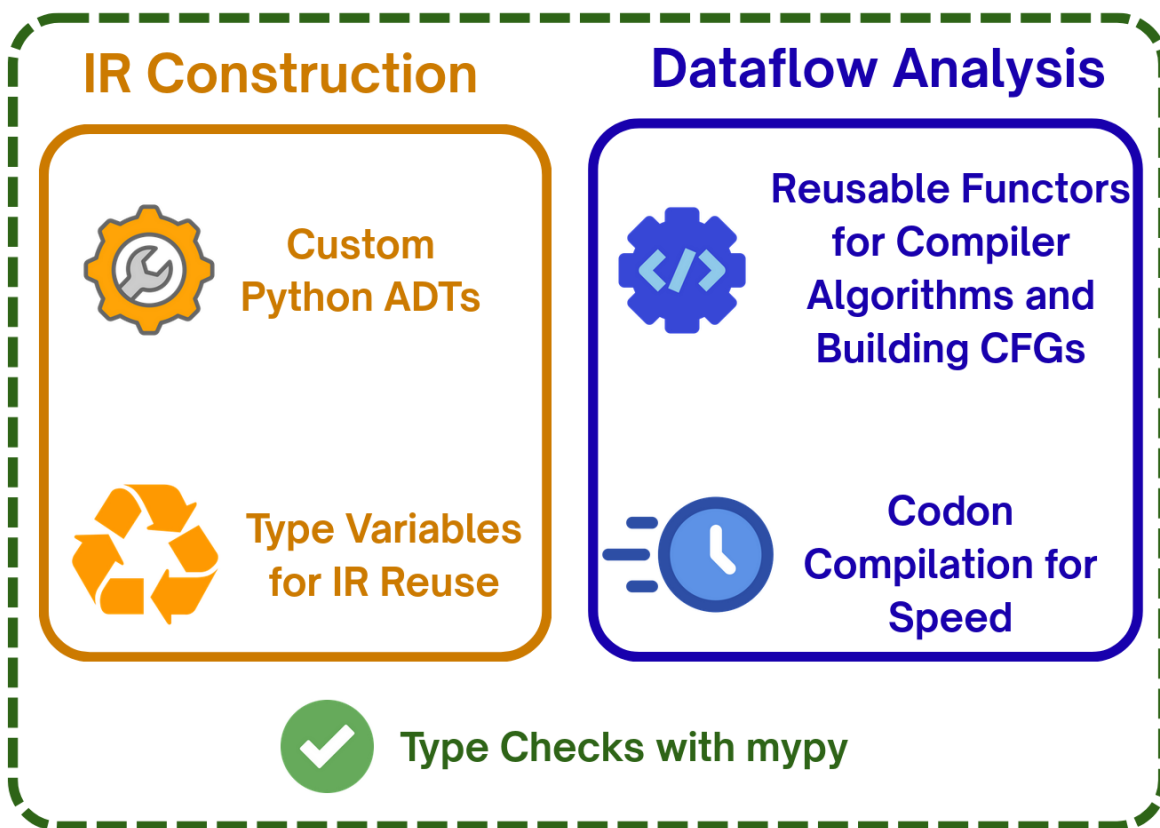


Figure 3.1: Overview of the two main components in the software created and their features. The IR construction provides custom types and reuse on the IR level. The dataflow analyses provide reusable functors for compiler algorithms and building CFGs and is partially compiled with Codon for speed

Note that our prototype is still in an early phase, and Table 3.1 shows the progress of each component. With the exception of the Codon integration, most of the components are built but require extra testing and development to be more robust.

In summary, our framework provides a few tools and functionality:

- Custom ADTs for Python [Chapter 4]

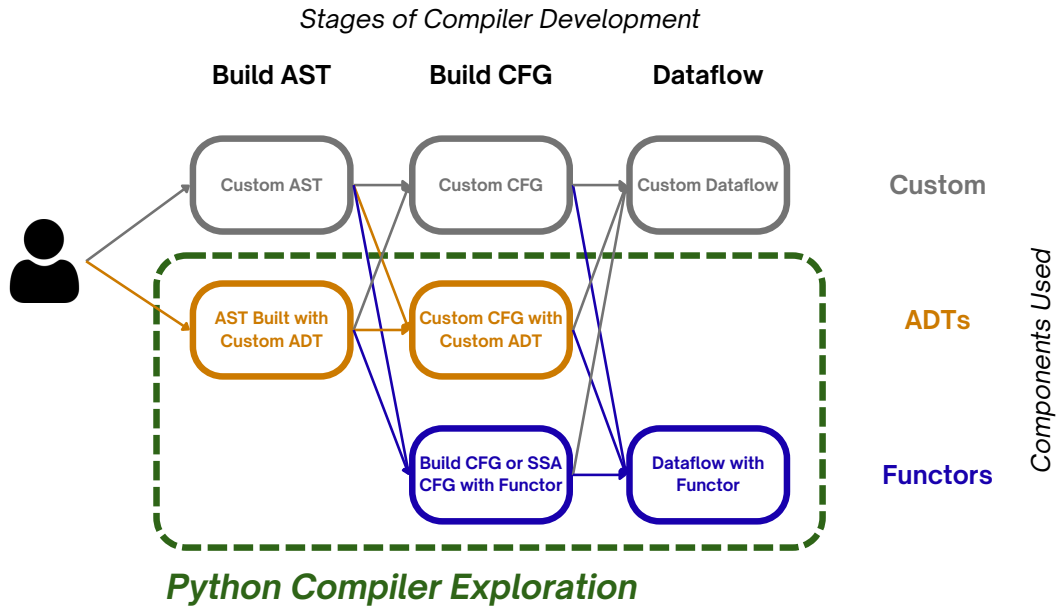


Figure 3.2: **Overview of ways to plug into our software to build a compiler.** Continuation from Figure 3.1. A summary of how the compiler exploration can be used in different stages of compiler development. The green box indicates features offered by our software and the arrows indicate any combination that the user may use. The Python compiler package provides flexibility, allowing users to use our custom ADTs and/or functors and/or use their own implementations. At any stage, a user can plug into our software.

Component	Successes	Limitations
ADTs	ADT mypy plugin ready-to-use	Highly dependent on the order so could be made more robust
Functors	Initial functors created for four compiler algorithms	Could be room for more generalization
Codon	Part of CFG functor compiled (at the cost of the type system) with a 10x speedup	Requires a JSON input, loses part of the type system, function inputs must be written in Codon

Table 3.1: Overview of the progress of each components

- Building a CFG (with options to make it SSA and/or perform dataflow analyses) from ADTs and functors [Chapter 5]
- Perform dataflow analyses on a custom CFG with functors specifying behavior [Chapter 5]
- Partial acceleration of these analyses with Codon [Chapter 6]

Chapter 4

Python ADTs

4.1 Chapter Summary

Defining and using (not to mention reusing) IRs in Python can be challenging due to the lack of type enforcement in Python. Python’s typing support contrasts popular compiler languages like C++ and OCaml which provide and enforce typing constructs like unions and generics. Consequently, this chapter explores add types helpful to compiler developers in Python.

This chapter will discuss creating a specific typing object, algebraic data types (ADTs), in Python. The two key typing features provided by this object are recursion and exhaustion. These are helpful compiler constructs to ensure that the compiler can model languages and covers all its cases.

Many programming languages have similar statements and expressions that we would like to reuse. Therefore, once we create the ADT, we are interested in how to reuse both the IR components and functions across the IR. In particular, we are interested in the following question: given an IR and a new node we want to add to the IR, what can we avoid changing? In this chapter, we explore how type variables in our Python ADTs facilitate some of this reuse. We further explore reuse of functions across the IR in [Chapter 5](#).

4.2 Abstract Data Types (ADT) Motivation and Design Goals

We provide a tool to creating abstract data types (ADTs) in Python. These have two main requirements: (1) the ADTs must explicitly support recursion in a way that can be type-checked and (2) the ADTs must support a completeness check.

Supporting recursion requires us to support ADTs such as code snippet 4.1. Code snippet 4.1 creates an ADT called `SuperType` which has elements `SubType1` and `SubType2`. Both of these elements can have attributes which are of type `SuperType`. This structure is common when building DSLs, for example with `Statement` ADTs in the IR which has attributes of type `Statement` which should be iterated over recursively. Listing 4.2 shows an example of a for loop which is a `Statement` but has a body which is also a `Statement`.

```
1 class SuperType:
2     pass
3
4 class SubType1(SuperType):
5     attr: SuperType
6
7 class SubType2(SuperType):
8     attr: SuperType
9
10 # Elements of SuperType ADT: SubType1 and SubType2
```

Listing 4.1: Outline of requirements for ADT

```
1 # the SuperType is a Statement
2 class Statement:
3     pass
4
5 # ForLoop is part of a Statement ADT
6 class ForLoop(Statement):
```

```

7     var: Variable
8     start: Expression
9     end: Expression
10    # notice: is recursive and has an attribute of type Statement!
11    body: Statement

```

Listing 4.2: Example of a recursive ADT element: a for loop!

The recursive syntax from 4.1 is supported in Python. However, we also want to be able to "close" the `SuperType` ADT or provide some method of checking completeness which continuing to support this recursion.

Python operates through an object-oriented interface via classes. However, there is no way to "close" a class, meaning for example 4.1, we would like for a match statement across an instance of the ADT to trigger a failure if one of its sub-elements are missing. For example, the failure could be triggered by typing extensions' `assert_never` as shown in 4.3. However, since Python does not understand the notion of closing a class, 4.3 does not type-check out-of-box.

```

1 supertype_adt_variable = SuperType()
2
3 match supertype_adt_variable:
4     case SubType1:
5         pass
6     case SubType2:
7         pass
8     case _:
9         # should be called if we are missing a case
10        assert_never()

```

Listing 4.3: Example of assert never

4.3 Python Limitations

Python currently supports dynamic types but not static types. While Python allows for static type annotations, it does not enforce these type annotations. Furthermore, Python lacks any notion of "closing" an ADT.

4.3.1 Python Type Checking Support

Static typechecking is a helpful feature for quickly identifying bugs and optimizing and writing readable code. Consequently, static type checking is an extremely nice property to have while writing compilers that isn't supported out-of-box by Python but is supported in languages such as C++ or ML style languages. Many developers want support for static types in Python, and there are many libraries providing options for static type checking in Python such as pyright [21], pyre [22], and mypy [23]. Of these, mypy is the most popular library at the time and supports integrations, so we decided to use it for this project. However, there are faster Python type checkers that are coming out and would be interesting to explore in the future. It has an internal representation of types in Python and can ensure that typing requirements are satisfied in a given Python file.

To get a notion of "closing" an ADT, we use mypy's extensions. Extensions have been used to allow for additional typechecking for special classes such as dataclasses and enums, so it is reasonable for adding additional typechecking notions for our custom ADTs.

4.4 mypy Background

```
1 # Command to execute mypy
2 mypy python_file.py
```

Listing 4.4: mypy execution command

mypy operates through running multiple passes across a Python file. As summarized in 4.1 consists of three main passes: (1) python parser, (2) semantic analyzer, and (3) type checker. The python parser creates the initial AST from Python code. Next, the semantic analyzer consists of two different types of analyses: a semantic analyzer and a type analyzer. Both of these analyses are intertwined and run iteratively in multiple passes of the program. Together, these attempt to generate constraints on the types of values in a Python program. The semantic analyze attempts to figure out how names relate to objects in the Python program while the type analyze attempts to use this context to fill in types. When they cannot completely decide on a type based on the generated constraints, they "defer" the type of a part of the program to resolved by a later use of either analysis. For example, if a semantic analyze pass allows us to later determine that a variable is created with a particular class constructor, we may only in a later type analyze pass be able to fill in the type of a member. Finally, the type checker validates that the constraints determined can be satisfied and the program can type check.

Pass	Description
Python Parser	Converts Python code to an AST using Python's AST module and converts that into a mypy-specific AST that can be accessed using mypy's visitor pattern
Semantic Analyzer	Binds names to definitions and performs consistent checks. The semantic analyzer may run multiple times across a portion of the AST to handle forward references and may hold "placeholder nodes" which are filled in during later passes.
Type Checker	Type checks the program and performs type inference. May also check parts of the AST multiple times to handle forward references.

Table 4.1: The three main mypy passes. The pass of most interest for developing Python ADTs is the semantic analyzer.

4.4.1 Semantic Analyzer Pass Details

Mypy provides the ability to extend the language through hooks which operate during the semantic analyzer step. Hooks will be discussed in more detail in 4.4.4. mypy has limited

documentation on the exact details of how the functionality of the semantic analyzer, so these overviews were derived based on reading the code and experimentation.

As previously mentioned, the semantic analyzer consists of `TypeAnalysis` and `SemanticAnalysis` passes. `TypeAnalysis` focuses on the analyzing the types, and `SemanticAnalysis` handles name binding, builds the symbol table, and performs some consistency checks.

When the semantic analyzer executes, mypy provides an initial `TypeAnalysis` pass followed by a `SemanticAnalysis` pass. Each pass evaluates the variables and types with different contexts. For example, since one pass of `TypeAnalysis` is run relatively early, it has a minimalist context, mainly just providing some metadata about the location of the object of question. In future passes such as the `SemanticAnalysis`, it includes more metadata such as attributes of the specific object including superclass information and class name.

4.4.2 defer

During the semantic analyzer pass, there is an option to `defer` the computation of a type. `Defer` will postpone the processing of the current node and trigger another pass of the semantic analyzer.

There is no documentation on the mypy developer docs on how exactly `defer` works. From code trials, we find that many code patterns will never allow for `defer` to resolve. One method we discovered that allows for `defer` to resolve is changing scope. For example, in a class, a `defer` from within the class can resolve after the semantic analyzer operates across the entire class.

4.4.3 mypy Types

mypy includes an abstraction of the type system represented internally. Of these, we identified a few key types which are relevant and used when developing DSLs. These types are shown in Table [4.2](#).

Type	Description
TypeVarLikeType	Type of a type variable
UnpackType	Unpacking * syntax
AnyType	Any
NoneType	None
DeletedType	Type of a deleted variable
FunctionLike	Python Functions
TupleType	Tuples
TypedDictType	Typed Dictionary
LiteralType	Literals
UnionType	Union
PartialType	Type with unknown type arguments
TypeType	Type variable

Table 4.2: mypy Types

4.4.4 mypy Hooks

mypy extensions operates through hooks which are activated at different passes of the semantic analyzer and modify the types. As summarized in 4.3, some hooks allow for modifying the type returned itself and some return other information in different types. Of all the hooks provided, only `get_type_analyze_hook` uses the type analyzer context, and the rest use the semantic analyzer context. The main difference between these two contexts is when it gets run and what metadata is provided. Generally, the type analyzer is run before the semantic analyzer and the semantic analyzer provides more metadata than the type analyzer.

4.5 Python Abstract Data Types

4.5.1 ADT Potential Design Choices

We have developed an implementation of ADTs using a mypy plugin which can statically verify completeness while supporting recursive types.

Before settling upon the final design, we iterated through several designs with different combinations of hooks from table 4.3. Since the completeness behavior works in mypy for

mypy hook	Returns type?	Used in ADT?	Analyzer Used
<code>get_attribute_hook</code>	✓	✓	SemanticAnalyzer
<code>get_base_class_hook</code>			SemanticAnalyzer
<code>get_class_attribute_hook</code>	✓		SemanticAnalyzer
<code>get_class_decorator_hook</code>			SemanticAnalyzer
<code>get_class_decorator_hook_2</code>			SemanticAnalyzer
<code>get_customize_class_mro_hook</code>			SemanticAnalyzer
<code>get_dynamic_class_hook</code>			SemanticAnalyzer
<code>get_function_hook</code>	✓		SemanticAnalyzer
<code>get_function_signature_hook</code>			SemanticAnalyzer
<code>get_metaclass_hook</code>			SemanticAnalyzer
<code>get_method_hook</code>	✓		SemanticAnalyzer
<code>get_method_signature_hook</code>			SemanticAnalyzer
<code>get_type_analyze_hook</code>	✓	✓	TypeAnalyzer

Table 4.3: mypy hooks with additional information on whether the hook returns a type that can be modified and whether they were used in the final ADT extension implementation

`UnionTypes`, our general approach was to replace an ADT with the union of all subtypes to get completeness checking in mypy.

To create this design, we needed two main features: (1) a way to collect all the subtypes and (2) a way to replace the supertype with the union of the subtypes.

One design choice we considered was having an explicit `Close` function of class to close the ADT. This design collected the subclasses using either `get_base_class_hook` or the `get_class_decorator_hook`. Both functioned more or less the same, we chose to use a base class since we preferred it aesthetically. The design next explored making use of the `get_function_hook` or `get_method_hook` to close the ADT. However, these calls were run after the type analysis pass, so it did not close the ADT soon enough. More precisely, future calls to the supertype did not yet have the closed ADT, so it could not replace it with the union.

To improve this implementation, we needed to make use of the hooks that were directly involved with the Type Analysis pass. We consequently instead opted to use the `get_type_analyze_hook` to directly modify the types and the `get_base_class_hook` to collect the subtypes. However, there still remained an issue: if a subtype had an attribute of the

supertype, how do we replace the supertype with the union (since the union is still being defined)?

To address this issue, we used mypy's defer function. However, it needed a nice point at which the defer could be resolved. Consequently, we used classes within a class. The class within a class style used provided a convenient point for the defer function to get resolved with the change in scope. Consequently, this format was pursued as opposed to adding special a Close function or class to close the ADT in which there was seemingly no way to resolve the defer to substitute an attribute of the class.

4.5.2 ADT Syntax

Our current syntax is shown in Listing 4.5 for a given `Super`, `Temp_ADT`, `A`, and `B` where `A` and `B` are members of the `Super` ADT.

```
1 class Super(ADT_Type):
2     pass
3
4 class Temp_ADT(ADT):
5     setup: Super
6
7     @dataclass
8     class A(ADT_Element, Super):
9         y: str
10        z: Super
11
12    @dataclass
13    class B(ADT_Element, SuperType):
14        pass
```

Listing 4.5: Current syntax for ADT. The ADT `Super` has members `Temp_ADT.A` and `Temp_ADT.B`

While the current syntax is a little verbose since it's built into the mypy hooks, this syntax provides static type checking of the ADTs and completeness checks which can more quickly catch bugs. Since it's static, it does not impact the execution of the Python program, including the runtime.

4.5.3 ADT Plugin Implementation Details

The ADT plugin is implemented using the `get_base_class_hook` and `get_type_analyze_hook`. Figures 4.1 summarizes how these hooks work together to construct the ADT, and Figure 4.2 demonstrates how the ADT can be used to typecheck downstream code. As described in Figure 4.1, when the `get_type_analyze_hook` is called for `adt.ADT`, we know that we are in an ADT. The first attribute of the ADT must be the super class of each element (for example, `setup: Super` in Figure 4.2). The following subtypes must inherit from `ADT_Element` and the super type (for example, `Super`) in that order. When `get_base_class_hook` is called with `ADT_Element`, we add it to a dictionary of the classes within the `Super` ADT. In addition, we flip a boolean when `get_type_analyze_hook` is called with `ADT_Element` to indicate to effectively ignore the call of `get_type_analyze_hook` on the following `Super`. However, if `Super` is called for an attribute (which can be identified by the lack of the boolean), we defer resolving the type till we have collected all subtypes to close the ADT.

Once the class is closed as indicated by a `get_base_class_hook` call to `ADT`, we have a list of the subtypes (in this example, `A` and `B`). `get_base_class_hook` is called, and we update the dictionary and reset. All the deferred types are replaced with the Union. At this point, we have processed the entire ADT and reached the end of Figure 4.1.

After the ADT has been created, we can typecheck downstream code like in Figure 4.2. The code demonstrates two instances of where `Super` should be replaced with the union of the subtypes: in the function header and in the attribute of the match statement. When `get_type_analyze_hook` encounters the `Super` type in the function header, it checks whether

the type exists in the ADT dictionary. If it exists, the super type is replaced with the union of the subtypes. Because mypy supports completeness checks for the union type, in the outermost match statement in Figure 4.2, the `assert_never` command will raise a mypy error when statically typechecked if the match statement is missing one of the supertype's subtypes.

We are now interested in supporting the completeness check recursively. Specifically, we would like to support a match statement within a match statement like in Figure 4.2. More precisely, in the match statement, we match a variable against subtypes which might have attributes of the supertype. We want these supertype attributes to be replaced with the union of the subtypes so we can also perform a completeness check. By the definition of our ADT, all of the attributes with type super type is already replaced with the union of the subtypes. Consequently, the hooks do not need to change any typing to get this feature: we the definition of our ADT, we can support embedded completeness checks. Consequently, the `assert_never` in the embedded match statement in Figure 4.2 will throw a mypy error if and only if the embedded match statement is missing a subtype.

4.6 Reuse

After creating the IR with our ADT, we are interested in how can reuse parts or all of an ADT. For example, creating DSLs of C, C++, and CUDA have a significant overlap. After creating a DSL for C, a user might want reuse both to (1) include IR components in the C++ ADT and (2) reuse functions that can operate across these components.

Recall Table 1.2 from the Introduction (Chapter 1). The table claim our framework gets reuse from both the AST level like staging and the IR level like MLIR. In this section, we address the reuse of IR components to get reuse at the AST level. We will discuss reusing functions on the IR via functors to get reuse at the IR level in Chapter 5.

Our framework provides reuse at the AST level by allowing users to add pre-defined IR

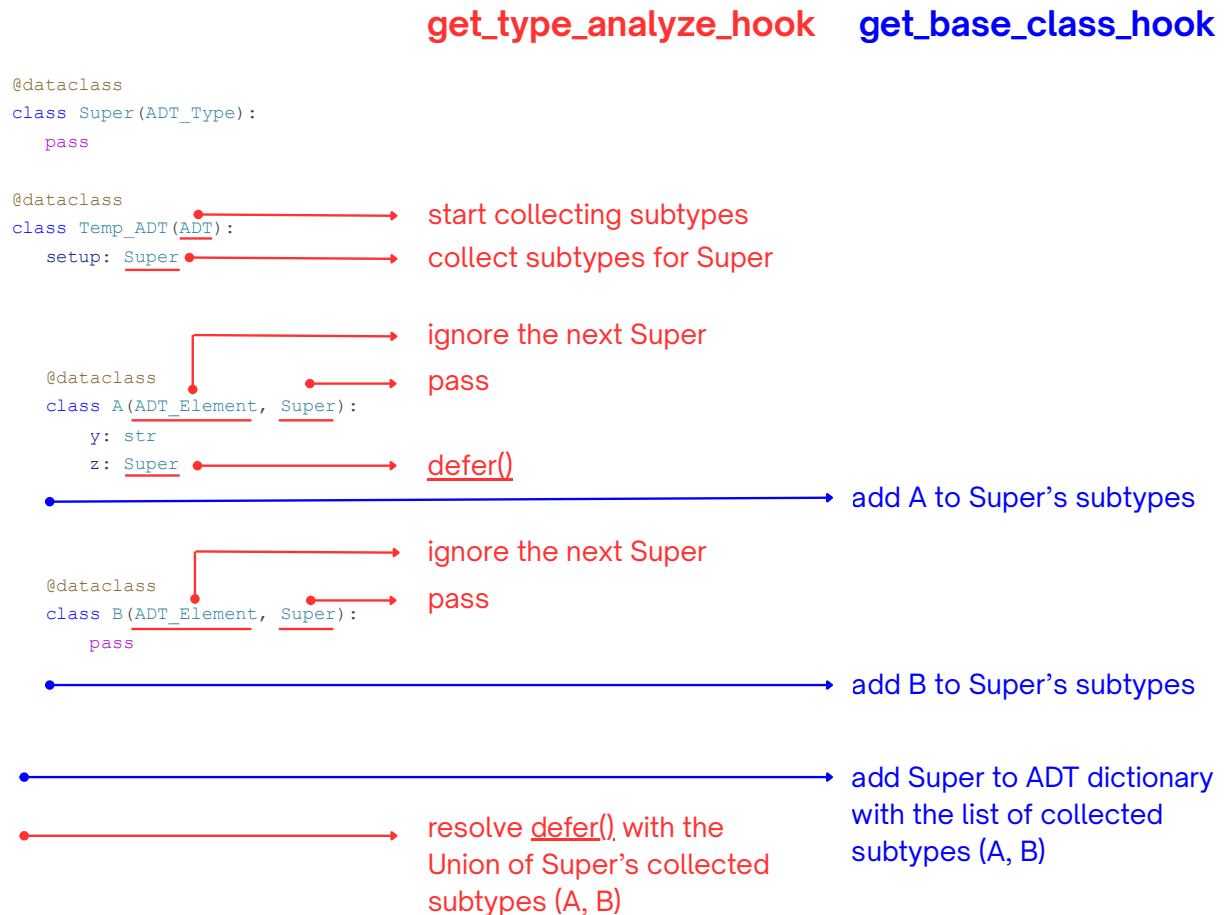


Figure 4.1: Breakdown of how an ADT invokes the mypy hooks to create Unions. See how it's used to ensure completeness in Figure 4.2

get_type_analyze_hook [no hook]

```
def foo(a: Super):
    match a:
        case Temp_ADT.A(y, z):
            match z:
                case Temp_ADT.A():
                    pass
                case Temp_ADT.B():
                    pass
                case _ as no:
                    assert_never(no)
            case Temp_ADT.B():
                ...
        case _ as no:
            assert_never(no)

a = Temp_ADT.A("example", Temp_ADT.B())
foo(a)
```

exists in ADT dictionary,
replaced with Union[A, B]

treated as Union[A, B]
(already replaced during
creation)

Figure 4.2: Continuation from Figure 4.1. Breakdown of how the ADT previously defined can be used for completeness and recursion

components to their custom defined ADT. This contrasts staging's method of providing reuse which is from a host language. Our method does not require users to "fight" the language to add custom features to the ADT but still allows users to reuse existing AST elements.

To allow for reusing of IR components, we support two main features: (1) using type variables to substitute language-specific types in ADT elements, and (2) allowing previously-defined ADT elements to be added to a new ADT.

4.6.1 Type Variables

Now we explore using type variables to reuse components of the IR. A user might create a node like the `Cond` (ternary) node in 4.6 for part of a C IR. As written, this definition's attributes contain direct references to a `CExpr` ADT and a `CType` ADT, components that are specific to the C DSL. If we want to reuse this component in a C++ ADT, we would instead want to use the `CppExpr` and `CppType` ADTs since C and C++ have slightly different expressions and types. To reuse the `Cond` node we already created, we can replace the C-specific ADTs with a type variable as shown in 4.7 that can be passed on definition of the ADT. This syntax now allows us to reuse the components since we can change the attributes' types depending on which ADT is being used.

```
1 class CExpressionADT():
2     ...
3
4     @dataclass(frozen=True)
5     class Cond(ADT_Element, CExpr[EAnn, TAnn]):
6         """Ternary conditional expression 'cond ? then : else'."""
7
8         exprCond: CExpr
9         exprT: CExpr
10        exprF: CExpr
11        ty: Optional[CType] = None
12
```

```
13     ...
```

Listing 4.6: CExpr without Type Variables

```
1 TAnn = TypeVar("TAnn")
2 EAnn = TypeVar("EAnn")
3
4 class CExpressionADT():
5     ...
6
7     @dataclass(frozen=True)
8     class Cond(ADT_Element, CExpr[EAnn, TAnn]):
9         """Ternary conditional expression ‘‘cond ? then : else‘‘."""
10
11         exprCond: EAnn
12         exprT: EAnn
13         exprF: EAnn
14         ty: Optional[TAnn] = None
15
16     ...
```

Listing 4.7: CExpr with Type Variables

4.6.2 Reuse Nodes within ADT

Now that we can support reusing nodes, we are interested in incorporating them into new ADTs one might build. We therefore additionally have support to reuse the component within an ADT without having to redeclare the IR component. For example, if a user wanted to use the `Cond` operation from their `CExpr` ADT in their `CppExpr` ADT, they can do so with the syntax shown in 4.8 and 4.9. This implementation uses inheritance of dataclasses to have all the same parameters and functions as the original component while redeclaring the base class to be `CppExpr` so it can be appropriately subclassed. In addition, it compiles with mypy and still has the property of completeness from Section 4.5.


```

1 @dataclass
2 class CExpr(ADT_Type, Generic[EAnn, TAnn]):
3     pass
4
5 class CExpressionADT(ADT, Generic[EAnn, TAnn]):
6     setup: CExpr[EAnn, TAnn]
7
8     ...
9
10 @dataclass
11 class Cond(ADT_Element, CExpr[EAnn, TAnn]):
12     """Ternary conditional expression ‘‘cond ? then : else‘‘."""
13
14     exprCond: EAnn
15     exprT: EAnn
16     exprF: EAnn
17     ty: Optional[TAnn] = None
18
19     ...

```

Listing 4.8: CExpr setup example

```

1 @dataclass
2 class CppExpr(ADT_Type, Generic[EAnn, TAnn]):
3     pass
4
5 class CppExpressionADT(ADT, Generic[EAnn, TAnn]):
6     setup: CppExpr[EAnn, TAnn]
7
8     ...
9
10 # minimal extra setup and will be part of the CppExpr ADT
11 @dataclass
12 class Cond(CExpressionADT.Cond[EAnn, TAnn], ADT_Element, CppExpr[EAnn,

```

```

    TAnn]):
13         pass
14
15     ...

```

Listing 4.9: CppExpr with reuse from CExpr

4.7 Additional Utilities

Along with implementing ADTs, we additionally implemented a few nice-to-have features with dataclasses in Python, namely dataclass type checking and improved dataclass hashing.

4.7.1 Dataclass Type Checking

One quirk of Python is it does not automatically check the types passed into a dataclass upon declaration. To address this limitation, we modified the dataclass implementation to assert whether a variable passed into the declaration of a dataclass is the correct type. This modification was done in the `dataclasses.py` file from Python, so as demonstrated in Listing 4.10, one can simply import our custom dataclass Python file and still use dataclasses as before. This dataclass file modifies the `__init__` function to automatically check that each of the dataclass fields are instances of the assigned type.

```

1 # custom dataclass importable like normal
2 from custom_dataclasses import dataclass
3
4 @dataclass
5 class Statement():
6     pass

```

Listing 4.10: Example of importing custom dataclass

After making these edits, we ensured our dataclass implementation is still compatible with mypy.

4.7.2 Dataclass Hashing

The current implementation of hashing for dataclasses in Python creates a string representation which is passed into a hashing function. This method is seemingly inefficient, so we developed an override for this hash function for our custom `ADT` and `ADT_element` types. This method assigns a prime number to each type as hashes the ADTs as a prime factorization of its types. This feature is also supported with the `custom_dataclasses` import above.

Memoization

As part of the dataclass hashing, dataclasses with the same attributes and types are mapped to the same location, allowing for memoization of the dataclasses. Memoization can help improve performance by reusing dataclasses we've created with the same structure.

Chapter 5

Functors

5.1 Chapter Summary

In the previous chapter, we achieved reuse at the IR-level through type variables. Now we are interested in reusing functions across the IRs; more precisely, we want a way to define compiler algorithms parametrically over IRs. We therefore draw on the idea of a functor and discuss how functors can be implemented in Python through type variables and functions. We then discuss the implementation of four compiler algorithms as functors: (1) CFG creation, (2) SSA CFG creation, (3) copy propagation with an SSA CFG, and (4) copy propagation with a CFG without SSA.

5.2 Motivation

ADTs can be used to easily build and express a language as an AST. Once this language is specified, we can perform a standard set of analyses and transformations to optimize the performance if provided with the relevant language semantics. These transformations are often dataflow analyses (or abstract interpretations) and include transformations like copy propagation and deadcode elimination. Together, dataflow analyses optimize code for improved runtime performance.

To allow for dataflow algorithms to be run with any program, we propose abstracting the dataflow analyses as functors which takes in the language and a few functions to specify the behavior of the language and can output an optimized version of the language. Functors can allow users to easily plug in dataflow analyses in Python for their DSL.

We provide functors for two main algorithms – constructing a control-flow graph and copy propagation. For both, we add an option for the control-flow graph to be in single-static assignment (SSA) form, leading to a total of four functors, as summarized in Table 5.2.

5.3 Functors

When we define functors, we refer to functors as used in languages like StandardML [24]. These are functions which are defined by mapping signatures (a collection of types and variables used to specify a structure) to signatures. When used, the functors map structures (a collection of type and values declarations) satisfying the signature. Structures are similar to a Python values/classes and signatures are similar to a collection of types used to define the class and functors can be maps from values to values typed as maps from a collection of types to another collection of types with the first group potentially parameterizing the second.

StandardML	Our Python Emulation
Signatures	Collection of types defining a class
Structures	Values/Classes
Functors (defined by mapping signatures to signatures, used to map structures to structures)	Maps from values to values types as maps from an input collection to an output collection with the input collection potentially parametrizing the output collection

Table 5.1: Comparison of StandardML’s functor implementation to our Python implementation

Functors can build in the flexibility to operate across any number of representations provided they satisfy some specific features (for example, a user provides functions that can extract information from the representation).

5.3.1 Our Implementation

Our emulation of functors consists of passing in type variables and constraining the properties of the type variables through additional functions that can be passed into the functor. The major compiler algorithm functions that will be expanded on in the next sections is summarized in Table 5.2.

Functor	Input	Result	Mutates Input?	Section
CFG Creation	ADT	CFG		5.4
SSA CFG Creation	CFG	SSA CFG	✓	5.5
SSA Copy Propagation	SSA CFG	SSA CFG	✓	5.6
Copy Propagation	CFG	CFG	✓	5.7

Table 5.2: Functors developed and discussed. Note that the functors that mutate the input return None and instead perform the tranformation on the input.

However, before exploring the more complex example, we will discuss a simple preliminary example: creating a functor to print a CFG. To allow for the flexibility to input any CFG definition, we start by creating a type variable for the CFG itself. As shown in Listing 5.1, create type variables by using `TypeVar` from Python’s `typing_extensions` library. The functor will return None since it’s just printing values. At this point, we have the start of the signature. To complete the signature, we need to add functions that operate across `CFGAnn`.

```
1 CFGAnn = TypeVar("CFGAnn")
2
3 def print_cfg(cfg: CFGAnn) -> None:
4     pass
```

Listing 5.1: Signature for a Print CFG Functor

To print the CFG, we want to iterate over it. CFGs consist of basic blocks which have instructions within it. This property can be defined as two functions that will constrain the behavior of the input: (1) a function to iterate over the basic blocks of the CFG and (2) a function to iterate across the instructions of the basic blocks. To maintain the generalization, we use type variables for basic blocks and instructions, as shown in Listing 5.2.

```

1 CFGAnn = TypeVar("CFGAnn")
2 BasicBlockAnn = TypeVar("BasicBlockAnn")
3 InsnsAnn = TypeVar("InsnsAnn")
4
5 def print_cfg(
6     cfg: CFGAnn,
7     get_basic_blocks_from_cfg: Callable[[CFGAnn], Iterable[BasicBlockAnn
8     ]],
9     get_instructions_from_basic_block: Callable[[BasicBlockAnn], Iterable[
10     InsnsAnn]]
11 ) -> None:
12     for basic_block in get_basic_blocks_from_cfg(cfg):
13         for insn in get_instructions_from_basic_block(basic_block):
14             # print the insn
15         pass

```

Listing 5.2: Print CFG Functor with helper functions to iterate through the basic blocks and instructions

Lastly, we want to print the instruction. We can require another function that converts the instruction into a string representation. Our completed functor is shown in Listing 5.3.

```

1 CFGAnn = TypeVar("CFGAnn")
2 BasicBlockAnn = TypeVar("BasicBlockAnn")
3 InsnAnn = TypeVar("InsnAnn")
4
5 def print_cfg(
6     cfg: CFGAnn,
7     get_basic_blocks_from_cfg: Callable[[CFGAnn], Iterable[BasicBlockAnn
8     ]],
9     get_instructions_from_basic_block: Callable[[BasicBlockAnn], Iterable[
10     InsnAnn]],
11     map_insn_to_str: Callable[[InsnAnn], str]
12 ) -> None:

```

```

11     for basic_block in get_basic_blocks_from_cfg(cfg):
12         for insn in get_instructions_from_basic_block(basic_block):
13             print(map_insn_to_str(insn))

```

Listing 5.3: Full functor for printing a CFG

We can now use this functor with any number of representations of CFG, BasicBlocks, and instructions. A simplified example of one such implementation is shown in Listing 5.4.

```

1 @dataclass
2 class Insn:
3     name: str
4     # + some extra info
5
6 @dataclass
7 class BasicBlock:
8     instructions: list[Insn]
9     # + some extra info
10
11 @dataclass
12 class CFG:
13     basic_blocks: list[BasicBlock]
14     # + some extra info
15
16 print_cfg(
17     CFG(basic_blocks=[
18         BasicBlock(instructions=[Insn(name="add"), Insn(name="sub")]),
19         BasicBlock(instructions=[Insn(name="mul"), Insn(name="div")])
20     ]),
21     get_basic_blocks_from_cfg=lambda cfg: cfg.basic_blocks,
22     get_instructions_from_basic_block=lambda bb: bb.instructions,
23     map_insn_to_str=lambda insn: insn.name
24 )

```

Listing 5.4: Simple implementation using the print CFG Functor

We can alternatively swap `Insn` for an ADT. The main edit needed to use the ADT is changing `map_insn_to_str` to, for example, a match function across the inputs. In addition, one might instead model the CFG as a linked list of basic blocks. To use this representation, one must change the `get_basic_blocks_from_cfg` to traverse the linked list. An example of using a linked list implementation instead is shown in Figure 5.5

```

1 @dataclass
2 class Insn:
3     name: str
4     # + some extra info
5
6 @dataclass
7 class BasicBlock:
8     _id: str
9     instructions: list[Insn]
10    successors: Optional[list["BasicBlock"]] = None
11    # + some extra info
12
13 @dataclass
14 class CFG:
15     entry_node: BasicBlock
16     # + some extra info
17
18 # traverse the linked list
19 def get_basic_blocks_from_cfg(cfg: CFG) -> Iterable[BasicBlock]:
20     visited = set()
21     stack = [cfg.entry_node]
22     while stack:
23         bb = stack.pop()
24         if bb._id not in visited:
25             visited.add(bb._id)
26             yield bb
27             if bb.successors:

```

```

28         stack.extend(bb.successors)
29
30 print_cfg(
31     CFG(entry_node=BasicBlock(_id="bb1", instructions=[Insn(name="add"),
32         Insn(name="sub")], successors=[
33         BasicBlock(_id="bb2", instructions=[Insn(name="mul"), Insn(name="
34         div")]))
35     ],
36     get_basic_blocks_from_cfg=get_basic_blocks_from_cfg,
37     get_instructions_from_basic_block=lambda bb: bb.instructions,
38     map_insn_to_str=lambda insn: insn.name
39 )

```

Listing 5.5: Simple implementation using linked lists for the print CFG Functor

The following sections will explore creating functors for more complex compiler algorithms.

Design Choices

Since the input is essentially a generic type with functions, one potential design choice is implementing functors with a Python Protocol, a rough equivalent to an interface. Protocols allow for structural subtyping or static duck typing: if a class has the same methods as the protocol defined, then it can be the type of the protocol. For example, if an instruction protocol had a method `map_insn_to_str` that inputs self and outputs a string, if a class had the same method with the same signature, it would satisfy the protocol.

Using protocols would lead to a much cleaner implementation since it doesn't need as many functions passed. However, protocols limit the flexibility: users are required to have specific functions and attributes. In addition, mypy struggles with typechecking protocols that use type variables and may not appropriately apply duck typing (for example, it may not match a function returning `str` to the function returning `InsnAnn`). Consequently, while passing in functions may look messier, they provide more flexibility for users and stronger

typechecking.

5.4 CFG Construction Algorithm

5.4.1 Algorithmic Overview

We now consider our first functor via the first step in a data flow analysis. One of the first steps when performing dataflow analysis is creating a control flow graph (CFG). CFGs consists of `BasicBlocks` with instructions and edges between them indicating control flow. We want to create a functor that is able to create a CFG from a given ADT along with functions containing metadata.

This algorithm is fairly straightforward: we start with an entry `BasicBlock` and add instructions that do not change the control flow path. When we reach an instruction that changes the control flow, we add a final instruction to the block that changes the control flow. This instruction is either a `Jump` instruction or a `Branch` instruction. We use a `Jump` instruction if there is exactly one location the control flow will change to. We use a `Branch` instruction if we have a condition that dictates which basic block the code will take. As shown in Figure 5.1, statements that can change dataflow include If/IfElse, While, and For loops, and the list of dataflow changes can also be extended to add other statements including `Continue` and `Break`.

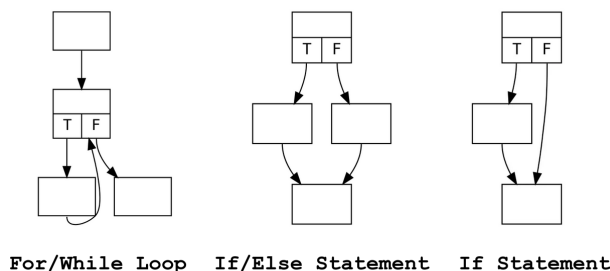


Figure 5.1: Example of common statements that change dataflow

5.4.2 Functor Representation

To abstract the creation of CFGs, we use a decision tree-like method based on function metadata provided to the functors, as described in Table 5.3. These are divided into overarching categories that may change control flow and more specific values can fall under these. For example, if the `Length` instruction is passed, it can be represented as a function and therefore a `CallInsn`. Similarly, `If` can be represented as a `IfElse` and `ForLoop` can be represented as a `WhileLoop`.

Helper Functions that Return non-None Values	Inferred Instruction
<code>get_expression && !get_destination_name</code>	<code>ReturnInsn</code>
<code>get_expression && get_destination_name</code>	<code>Assign (LoadConstInsn or CopyInsn)</code>
<code>get_expression_list && get_destination_name</code>	<code>CallInsn</code>
<code>get_statement_list</code>	<code>BlockStmt (recurse)</code>
<code>get_condition && !is_loop</code>	<code>IfElseStmt (Branch)</code>
<code>get_condition && is_loop</code>	<code>WhileLoop (Branch)</code>
<code>get_workflow_change</code>	<code>Break/Continue</code>

Table 5.3: The mapping used to identify the corresponding instruction and/or statement based on input functors when building the CFG

These functions take in a statement and output the necessary information. For example, for getting the condition, the function will return the `condition` attribute if applicable as shown in Listing 5.6.

```

1 def get_condition(stmt: AssemblyStatement) -> Optional[AssemblyExpression
    ]:
2     if (
3         isinstance(stmt, If) or
4         isinstance(stmt, IfElse) or
5         isinstance(stmt, WhileLoop)
6     ):
7         return stmt.condition
8     return None

```

Listing 5.6: Get Condition Helper Function

In addition to the above metadata, the build CFG also takes in additional helper functions to both create the CFG and access attributes of the basic blocks as detailed in Listing 5.7. Notice that all of the parameters passed are type annotations except the instruction ADT which has pre-built instructions. This implementation could be further generalized by instead passing an equivalent `create_X_insn` helper functions; however, at that point, the amount of generalization gained is limited.

```

1 ASTAnn = TypeVar("ASTAnn")
2 StmtAnn = TypeVar("StmtAnn")
3 ExprAnn = TypeVar("ExprAnn")
4 BBAnn = TypeVar("BBAnn")
5 CFGAnn = TypeVar("CFGAnn")
6
7 def build_cfg_from_ast(
8     ast: ASTAnn,
9     init_cfg: Callable[[], CFGAnn],
10    init_basic_block: Callable[[], BBAnn],
11    add_basic_block_to_cfg: Callable[[CFGAnn, BBAnn], None],
12    get_statements: Callable[[ASTAnn], Iterable[StmtAnn]],
13    set_entry_block: Callable[[CFGAnn, BBAnn], None],
14    set_exit_block: Callable[[CFGAnn, BBAnn], None],
15    # notice: the only fixed variable: Insn
16    add_insn_to_block: Callable[[BBAnn, Insn], None],
17    add_successor: Callable[[BBAnn, BBAnn], None],
18    # [helper functions for identifying statements]
19 ) -> CFGAnn:
20     ...

```

Listing 5.7: Type Signature of the build CFG function

5.5 CFG to SSA CFG

Once the CFG is created, we can convert it to a single-static assignment (SSA) CFG. SSA is a program representation where each variable is assigned exactly once. It is used in most modern-day compilers as its property of having exactly one assignment makes dataflow analyses easier by abstracting away the complexity of redefinitions along dataflow.

Our goal is to make the SSA algorithm into a reusable and typed compiler algorithm in Python. Since the input to SSA is a CFG, we want to support most representations of a CFG, not necessarily just the CFG generated in Section 5.4. Furthermore, since the CFG creation algorithm in Section 5.4 takes an input most representations of a language, we can chain the two algorithms together to generate an SSA CFG of most language representations.

There are a number of algorithms to perform an SSA conversion, and we chose to follow the simple algorithm outlined in the SSA book [25]. High-level, the SSA algorithm works by calculating the dominance frontier to perform live-range-splitting with phi nodes and then renames the variables to be defined only once.

5.5.1 Dominators

Dominators and Immediate Dominator

We firstly calculate the dominators and immediate dominator of each block in the CFG. For a given block b , dominators are blocks such that all flows through b also goes through it and the immediate dominator is the closest block c such that all paths that go through b also go through c .

Dominance Frontier

The next calculation needed for phi node insertion is getting the dominance frontier. The dominance frontier are blocks b such that b dominates an immediate predecessor of a certain

block but does not immediately dominate that block. This algorithm is described by Listing 5.8 below.

```
1 for a, b in cfg_edges:
2     x = a
3     # while x does not strictly dominate b
4     while not strictly_dominates(x, b):
5         DF[x] &= b
6         x = immediate_dominator(x)
```

Listing 5.8: Dominance Frontier Algorithm

5.5.2 Phi Node Insertion

With the calculated dominance frontier, we can insert phi nodes as described by the algorithm 5.9 below. For each variable, the algorithm essentially adds phi nodes to the basic blocks on the dominance frontier of blocks with a definition of a given variable.

```
1 for v in variable_names:
2     # set of basic blocks where  $\phi$  is added
3     F = set()
4     # set of basic blocks that contain definitions of v
5     W = set()
6
7     for B in get_basic_blocks_with_definition(v):
8         W &= B
9
10    while len(W) > 0:
11        X = W.pop()
12
13        for Y in DF[X]:
14            if Y not in F:
15                add_phi_node(v, Y)
16                F &= Y
```

```

17         if Y not in get_basic_blocks_with_definition(v):
18             W &= Y

```

Listing 5.9: Phi Node Insertion Algorithm

This function requires a few functions, including (1) a function to take a CFG and return all variable names, (2) a function to get all the basic blocks where a variable is defined, and (3) a function that will add phi nodes to a basic block. Note that in our implementation and more generally, phi nodes are handled separately than instructions and have their own functions to handle them.

5.5.3 Variable Renaming

Once the phi nodes are established, the final step is renaming the variables such that every variable is defined exactly once (but can possibly be used multiple times). Along with the previous functions, this algorithm additionally needs functions to update the variable defined and iterate through the variables used.

This algorithm introducing the idea of a reaching definition which essentially is a previous definition that is defined in a block dominating the current block. This idea of a reaching definition is used in dataflow analyses like copy propagation.

```

1 reachindDef = dict()
2
3 for v in get_variables(cfg):
4     reachingDef[v] = None
5
6 for BB in DFS_preorder(cfg):
7     for i in get_instructions(BB):
8         for v in get_variables_used(i):
9             updateReachingDef(v, i)
10            # replace use of v with reachingDef[v] in i
11            replace_var(v, reachingDef[v], i)
12

```



```

13     for v in get_variables_defined(i):
14         updateReachingDef(v, i)
15         v_p = new_variable()
16         replace_var(v, v_p, i)
17         reachingDef[v_p] = reachingDef[v]
18         reachingDef[v] = v_p
19
20     for phi in get_phi_insn_in_successors(bb):
21         for v in get_variables_used(phi):
22             updateReachingDef(x, phi)
23             replace_var(v, reachingDef[v], phi)

```

Listing 5.10: Variable Renaming Algorithm

```

1 r = reachingDef[v]
2
3 while not (r is None or dominates(definition(r), i)):
4     r = reachingDef[r]
5
6 reachingDef[v] = r

```

Listing 5.11: Update Reaching Definition Algorithm

5.5.4 SSA Example

An example SSA CFG generated is shown in Figure 5.2. The CFG is from a simple if/else statement from 5.12. Since there are two paths, blocks 2 and 4 are not dominated by each other and are therefore in each other's dominance frontier, so we insert three phi nodes in block 3. Note that a more sophisticated SSA algorithm may not add the phi node for `t5` and `t6`. With the variable renaming algorithm, each variable is renamed appropriately, and the phi nodes are also appropriately updated.

```

1 def ifelse():
2     a = 5

```

```

3  if a == 5:
4      a = 10
5  else:
6      a = 20
7  return a

```

Listing 5.12: Simple If/Else for Demonstration CFG

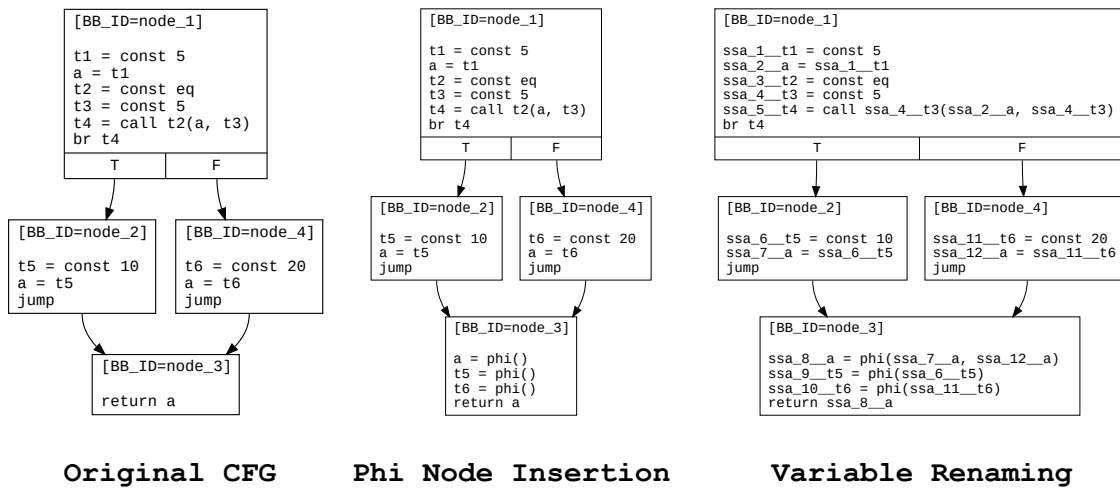


Figure 5.2: Example of all of the stages of SSA on an if/else statement

5.5.5 SSA CFG Functor

The functor for SSA CFG is shown in 5.13. When creating the SSA algorithm into a functor, we want to support a CFG generated from the previous functor in Section 5.4 along with a user's custom-defined CFG that meets some constraints. Similarly, we want to satisfy both our instruction ADT along with custom instruction inputs. In addition, we need extra functions to deal with phi nodes. We make the assumption that variable definitions are defined with strings – variable definitions could be parametrized with a type variable but is

generally a reasonable assumption. Finally, all internal components like the dominators and dominance frontiers are not parametrized by a type variable since it's internal and we can safely use our custom representations.

```

1 CFGAnn = TypeVar("CFGAnn")
2 BBAnn = TypeVar("BBAnn")
3 # id MUST be hashable for reasonable performance
4 BBIDAnn = TypeVar("BBIDAnn", bound=Hashable)
5 StmtAnn = TypeVar("StmtAnn")
6 ExprAnn = TypeVar("ExprAnn")
7 InsnAnn = TypeVar("InsnAnn")
8
9 def do_ssa(
10     cfg: CFGAnn,
11     # accessing the CFG
12     get_basic_blocks: Callable[[CFGAnn], Iterable[BBAnn]],
13     get_entry_node: Callable[[CFGAnn], BBAnn],
14     get_bb_id: Callable[[BBAnn], Hashable],
15     get_successors: Callable[[BBAnn], Iterable[BBAnn]],
16     get_predecessors: Callable[[BBAnn], Iterable[BBAnn]],
17     get_insns: Callable[[BBAnn], Iterable[InsnAnn]],
18     # adding phi nodes
19     create_phi_node: Callable[[str], InsnAnn],
20     add_phi_node_to_block: Callable[[BBAnn, InsnAnn], None],
21     get_phi_nodes: Callable[[BBAnn], Iterable[InsnAnn]],
22     # additional helper functions
23     get_variables_used_by_instruction: Callable[[InsnAnn], Iterable[str]],
24     get_variables_defined_by_instruction: Callable[[InsnAnn], Iterable[str]
25     ]],
26     replace_definition_with_new_var: Callable[[str, str, InsnAnn], None],
27     create_fresh_variable: Callable[[str], str],
28     get_variables_used_by_phi: Callable[[InsnAnn], Iterable[str]],
29 ) -> None:

```

Listing 5.13: Signature of SSA algorithm

5.6 Copy Propagation (with SSA)

After generating the CFGs, we next want to perform dataflow analyses over them to optimize the code generated. Copy propagation is a common dataflow analysis that a user might want to use in their compilers. Therefore, we want to implement the copy propagation algorithm to be reusable and typed in Python. We want to allow for users to both use an SSA CFG we created or one of their own design. This section explores implementing this transformation as a functor.

5.6.1 Copy Propagation Definition

Copy propagation is a dataflow optimization that directly substitutes copies of a variable. For example, the code in [5.14](#) directly substitutes the copy in line 1 into line 2 to become the code in [5.15](#).

```
1 x = y
2 z = x + 1
```

Listing 5.14: Sample Code Pre-Copy Propagation

```
1 x = y
2 z = y + 1 # replace x with y directly
```

Listing 5.15: Sample Code Post-Copy Propagation

The extra `x = y` becomes unnecessary and can be eliminated in later transformations like deadcode elimination.

5.6.2 SSA Copy Propagation

One benefit of SSA is it allows for dataflow analyses to be notably easier. One of such examples is copy propagation. Since every value is assigned at most once, with an SSA CFG, one does not need to worry about killing copies of a variable when a variable is reassigned. In addition, changes in variables along different paths is handled by the phi node. Consequently, one can simply traverse the CFG and replace copies directly.

5.6.3 SSA Copy Propagation Functor

The copy propagation functor signature is shown in 5.16. This implementation once again assumes variables are strings and provides the appropriate helper functions and mapping functions.

```
1 def copy_prop_ssa(  
2     cfg: CFGAnn,  
3     get_basic_blocks: Callable[[CFGAnn], Iterable[BBAnn]],  
4     get_insns: Callable[[BBAnn], Iterable[InsnAnn]],  
5     get_copy_insn_dest_and_src: Callable[[InsnAnn], Optional[Tuple[str,  
6         str]]],  
7     replace_use_with_dict: Callable[[InsnAnn, dict[str, str]], None]  
8 ) -> None:  
9     ...
```

Listing 5.16: SSA Copy Propagation Functor Signature

5.7 Copy Propagation (without SSA)

The final functor we present is copy propagation without SSA. We once again want to provide this algorithm as a reusable and typed transformation that can operate across most CFG representations since it is a commonly used compiler algorithm.

5.7.1 Algorithm Overview

If we want to perform copy propagation without SSA, we need to be more cautious of variables being redefined. Copy propagation without SSA can be divided into local copy propagation and global copy propagation.

Local copy propagation is contained within a basic block and is similar to the SSA copy propagation. One can iterate through the instructions, and if an instruction is a copy, one can store that copy to be used to replace future references. Different from SSA, however, is that there can be re-definitions within a block. If a re-definition is encountered, the older replacement must be killed/replaced.

To make copy propagation global, we want to only replace a copy if the replacement dominates a node and there is no redefinition. More precisely, we consider the input copies and the output copies for a given basic block. The block's predecessors' copies are its input copies. To combine these inputs, we take the intersection to ensure that all copies are attainable from any path. Since copy propagation is an iterative algorithm, if we haven't yet reached an input node (for example, in a loop), assume that it has the set of all possible copies. The output is all local copies plus the input copies minus the killed copies. Note that to make a copy unique, we include the source and destination along with a unique location (ex: basic block id and line number). We iterate till we reach a fixed point.

Similar to SSA, the main functions that are necessary for copy propagation is (1) identifying definitions of variables, (2) identifying uses of variables, and (3) modifying the uses of instructions. These functions can be reused between the SSA implementation.

An example showing both global and local copy propagation is shown in [Figure 5.3](#).

5.7.2 Functor Overview

The extra definitions for copy propagation without SSA is similar to SSA copy propagation and shown in [5.17](#).

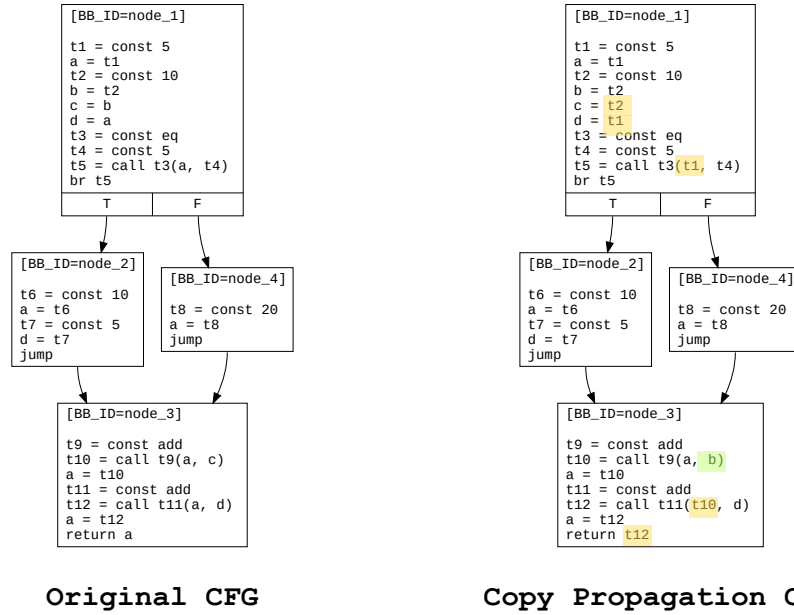


Figure 5.3: Example of copy propagation on a non-SSA CFG. The yellow highlights indicate local copy propagation and the green highlight indicates global copy propagation.

```

1 def copy_prop(
2     cfg: CFGAnn,
3     get_basic_blocks: Callable[[CFGAnn], Iterable[BBAnn]],
4     get_insns: Callable[[BBAnn], Iterable[InsnAnn]],
5     get_bb_id: Callable[[BBAnn], str],
6     get_copy_insns: Callable[[InsnAnn], Optional[Tuple[str, str]]],
7     replace_use_with_dict: Callable[[InsnAnn, dict[str, str]], None],
8     get_definition: Callable[[InsnAnn], Optional[str]],
9     get_successors: Callable[[BBAnn], Iterable[BBAnn]]
10 ) -> None:
11     ...

```

Listing 5.17: Copy Propagation Functor Signature

Chapter 6

Codon Compatibility

6.1 Chapter Summary

One major drawback of writing compilers in Python is that Python is slow. To make our pipeline faster, we exploring ways to compile our Python code, specifically the functors, with Codon. We aimed to improve the performance of our pipeline and maintain our reuse and typing, but this was not entirely possible in the current version of Codon. This section explores the modifications needed to convert our abstractions into Codon and shows which components from our pipeline could and could not be maintained.

6.2 Codon Motivation and Background

The previous work from the previous sections provides helpful abstractions to use while developing compilers in Python. However, one major limitation of Python and any interpreted language is that it's much slower than compiled languages like C. To address this, developers have created Python compilers that allow for notable Python runtime improvements.

Some examples of Python compilers include Numba [26], MyPyC [23], PyTorch [27], Pypy [28], Cython [29], and Codon [20]. For this project, we chose to use Codon since it has a thorough coverage of the Python programming language and has shown strong improvements

to Python’s runtime.

Codon is a Pythonic DSL with syntax similar to Python/CPython. Furthermore, it uses ahead-of-time type checking during compilation. It aims to provide a highly-performance high-level interface that compiles Python-like code to be at least on par with C or C++ performance.

Codon provides three methods of compilation: (1) `codon build` and `codon run` which compiles Codon into an executable, (2) PyExt which compiles Codon into Python extension, and (3) Codon JIT compilation that can be directly used in Python. The successes and issues with each method is summarized in 6.1 and will be discussed in detail throughout the chapter.

Codon Method	Issues	Success
<code>codon run</code>	Needs Python string input/output, function inputs must be written in Codon	Maintains most of the type system
<code>pyext</code>	Doesn’t support polymorphic types (lose all types)	proof-of-concept compiled (no typing)
<code>jit</code>	Needs Python string input, ADT syntax not supported	proof-of-concept with part of the type system

Table 6.1: Codon Compilation Issues and Successes Summary

6.2.1 Speedup Expectations

Codon claims a 10x-100x speedup of Python code. For the compiler algorithms we implemented, we also expect a similar speedup. Most compiler algorithms are fixed-point algorithms and recursive across structures like CFGs and ADTs. The recursive structure can likely improve from Codon compilation through optimizations Python currently does not support like tail recursion (tail call optimization). Furthermore, there isn’t anything special about fixed-point algorithms in compilers when it comes to low level performance, so we expect simply removing the interpreter overhead to be the biggest win. More generally, when we implement compiler algorithms in languages such as C/C++, we do not use their hardware oriented features to accelerate the algorithm except maybe for parsing. Thus, the

main difference really ought to be the overhead of interpreting and disable general purpose optimizations that get enabled.

6.3 Codon Integration End Result

To get our pipeline to compile in Codon, we had to sacrifice many of our features. In this section, we describe our final interface and in the following sections, we describe the details of the implementation and relevant design choices.

We implemented the AST to CFG functor in Codon. Note that this was developed in Codon which is not entirely Python compatible (explored in Section 6.5). We have developed several functions that exported our ADT in Python into JSON which is read in Codon and converted back into a slightly ADT. After the CFG conversion is generated, it is also exported as JSON to be read by Python.

Since the input was passed through JSON and Codon does not support Python's `eval` function, it was difficult to send a function. Consequently, we required all metadata functions passed to the functors to be defined in Codon. Similarly, as discussed in Section 6.4, Codon does not support type variables, so the inputs to the functors were a fixed type. Together, these limited the flexibility offered. Listing 6.1 shows the original definition of a functor and Listing 6.2 shows the new definition in Codon.

```
1 # from Python
2 def build_cfg_from_ast(
3     ast: ASTAnn,
4     init_cfg: Callable[[], CFGAnn],
5     init_basic_block: Callable[[], BBAnn],
6     add_basic_block_to_cfg: Callable[[CFGAnn, BBAnn], None],
7     get_statements: Callable[[ASTAnn], Iterable[StmtAnn]],
8     set_entry_block: Callable[[CFGAnn, BBAnn], None],
9     set_exit_block: Callable[[CFGAnn, BBAnn], None],
10    # notice: the only fixed variable: Insn
```

```

11     add_insn_to_block: Callable[[BBAnn, Insn], None],
12     add_successor: Callable[[BBAnn, BBAnn], None],
13     # [helper functions for identifying statements]
14 ) -> CFGAnn:
15     ...

```

Listing 6.1: Functor in Python

```

1 # from Codon
2 def build_cfg_from_ast(
3     ast: AST,
4     # [helper functions for operations across the AST and CFG not needed
4     #   due to fixed implementation]
5     # [helper functions for identifying statements pre-defined in Codon]
6 ) -> CFG:
7     ...

```

Listing 6.2: Functor in Codon

However, we could maintain the ADT that is type-checkable with mypy (though note that Codon in general isn't supported by mypy) with a few modifications as described in Section 6.4.1. The original syntax is shown in Listing 6.3, and the Codon syntax which is more verbose but still compatible with our ADT definition is shown in Listing 6.4. Also note that since the code is read through JSON, the user can still use the Listing 6.3 in their Python code which is automatically converted to Listing 6.4 in the Codon code.

```

1 @dataclass
2 class AssemblyExpression(ADT_Type):
3     pass
4
5 class Expression_ADT(ADT):
6     setup: AssemblyExpression
7
8     @dataclass

```

```

9     class Literal(ADT_Element, AssemblyExpression):
10         val: Any
11
12     ...

```

Listing 6.3: ADT in Python

```

1 @dataclass
2 class AssemblyExpression(ADT_Type):
3     def get_val(self) -> str:
4         a: str = ""
5         return a
6
7     ...
8
9 class Expression_ADT(ADT):
10     setup: AssemblyExpression
11
12     @dataclass
13     class Literal(ADT_Element, AssemblyExpression):
14         val: str
15
16         def get_val(self):
17             return self.val
18
19     ...

```

Listing 6.4: ADT in Codon

In summary, the changes included:

- **I/O:** Data is passed in and out of our Codon implementation through JSON
- **ADTs:** Our ADTs still type check (if was written in Python) but has slightly different syntax

- **Functors:** The functor has a fixed input and output type (no type variables) and all input functions must be written in Codon. The algorithm itself operates as expected with a notable runtime improvement.
- **mypy Compatibility:** Codon is not compatible with mypy and instead handles type checking itself when compiling.

6.4 Codon Compilation Of Our Compiler Infrastructure

We started by specifically looking at compiling the function to convert an ADT to a CFG.

Codon only supports certain libraries and features which unfortunately does not include `TypeVar`. This already limits the amount of reuse we can get from the functors since it forces a user to use our implementation. This could be later addressed by creating a Python functor to convert, for example, an arbitrary format of a CFG to our CFG definition.

Since we could not use type variables, we started to explore compiling with Codon by just compiling a typed functor without the type variables. In other words, we used a specific type for input language and a specific type for the CFG language. We faced challenges compiling just that base implementation, and this chapter discusses the challenges and workarounds.

To allow for the pipeline to be compiled with Codon (`codon build`), we had to make a handful of changes to the original script, the most notable changes being with the type system.

6.4.1 Accessing Attributes

One of the first challenges we encountered was accessing attributes in Codon. Code involving accessing attributes of subclasses that compiles fine with Python but not in Codon is shown in Listing 6.5. This example shows how if a parent class does not have an attribute that the child class has and the type a function takes in is the parent class, we will not be able to access the child class's attribute even if the class passed has the attribute.

```

1 @dataclass
2 class AssemblyStatement:
3     pass
4
5 @dataclass
6 class Assign(AssemblyStatement):
7     lhs: AssemblyExpression
8     rhs: AssemblyExpression
9
10 def walk_ast(stmt: AssemblyStatement):
11     # must use chained if/elif since match across a type is not supported
    with Codon
12     if isinstance(stmt, Assign):
13         # does not compile since AssemblyStatement does not have lhs/rhs
14         lhs = stmt.lhs
15         rhs = stmt.rhs

```

Listing 6.5: Subclassing that fails in Codon

Initial Attempts of Accessing Attributes

To make accessing subclass attributes compile, we tried two initial approaches to provide an optional attribute: adding a `Union` with `NoneType` to the super class (Listing 6.6) and making the attribute `Optional` (Listing 6.7).

```

1 @dataclass
2 class Statement:
3     bodies: list[str] | NoneType = None
4     var: str | NoneType = None
5     val: Statement | NoneType = None
6
7 @dataclass
8 class Block(Statement):
9     bodies: list[str] | NoneType

```

```

10
11 @dataclass
12 class Assign(Statement):
13     var: str | NoneType
14     val: Statement | NoneType
15
16 # fails to compile: union already sealed

```

Listing 6.6: NoneType Error

```

1 @dataclass
2 class Statement:
3     bodies: Optional[list[str]] = None
4     var: Optional[str] = None
5     val: Optional[Statement] = None
6     # attributes never changes from None
7
8 @dataclass
9 class Block(Statement):
10     bodies: Optional[list[str]]
11
12 @dataclass
13 class Assign(Statement):
14     var: Optional[str]
15     val: Optional[Statement]

```

Listing 6.7: Codon Annotation with Optional

However, both of these methods fails. Listing 6.6 failed with `union already sealed`. From trial-and-error, Codon generally runs into less compilation errors when using `Optional` than when unioning with `NoneType`. However, while Listing 6.7 does compile, the code did not have the correct functionality. It failed since the value set never changing from `None`.

Final Implementation of Accessing Attributes

An implementation that compiled with Codon and behaved as expected was creating get and set methods, as shown in Listing 6.8. This implementation had the superclass contain get and set functions with the expected signature which returned a dummy value. Through use of `isinstance` and `assert False`, we expect the dummy value to never be reached, but Codon needs the functions to return the correct type to compile. Each subclass has its own local attributes and overrides their corresponding get and set methods with accessing these local attributes.

```
1 @dataclass
2 class AssemblyStatement:
3     def get_lhs(self) -> AssemblyExpression:
4         assert False
5         return AssemblyExpression()
6
7     def get_rhs(self) -> AssemblyExpression:
8         assert False
9         return AssemblyExpression()
10
11 @dataclass
12 class Assign(AssemblyStatement):
13     lhs: AssemblyExpression
14     rhs: AssemblyExpression
15
16     def get_lhs(self) -> AssemblyExpression:
17         return self.lhs
18
19     def get_rhs(self) -> AssemblyExpression:
20         return self.rhs
21
22 def walk_ast(stmt: AssemblyStatement):
```



```

23     # must use chained if/elif since match across a type is not supported
    with Codon
24     if isinstance(stmt, Assign):
25         # compiles!
26         lhs = stmt.get_lhs()
27         rhs = stmt.get_rhs()

```

Listing 6.8: Subclassing that works in Codon

6.4.2 `isinstance` and `Any` type

Furthermore, certain typing schemas result in `isinstance` failing to return the correct value, sometimes thinking that an input is only, for example, a `pyobj` (Codon’s catch-all for an unknown Python type) even though it’s also an `Assign` statement. To ensure that `isinstance` always works, we set up the typing system so the types would always resolve which required making sure all types were exactly the same and avoided `Optional` and `Any` types which might cast the type passed into a different type and would be difficult to resolve.

In general, the use of `Any` should be avoided since it is difficult to resolve `Any` to a specific type. For example, if one has a variable of type `Block` and passes it into a function which takes in variable `var` of type `Any`, `isinstance(var, Block)` may not return `True`. There exists hacky workarounds like accessing a string attribute to help with type casting, but we tried to avoid that when possible.

Furthermore, we tended to stay away from the `Any` type since it was difficult to parse the `Any` back into a JSON since `isinstance` wouldn’t work on `Any` types. Even getting a string print-out of an `Any` type could result in the string `Any`. Consequently, we changed all the `Any` attributes in our `Statement`, `Expression`, and `Instruction` ADTs (the ADTs that were being read in and out) into strings which were evaluated back into its particular type (ex: `float`, `int`, etc.) when executing the CFG.

Listing 6.9 shows a similar type casting error which works in Python but not in Codon.

In Codon, if one were to have a function take in type `AssemblyStatement`, it retains information about subtype. For example, if one were to pass `Assign`, a class that inherits from `AssemblyStatement`, `isinstance(stmt, Assign)` will return `True`. However, if one were to pass that value into a new function's field of type `Optional[AssemblyStatement]` as the parameter, Codon may lose the information of the subtype, so checking the same `isinstance(stmt, Assign)` from the new function will return `False`.

```
1 def optional_stmt(stmt: Optional[AssemblyStatement]) -> bool:
2     # will always be False
3     if isinstance(stmt, Assign):
4         return True
5     else:
6         return False
7
8 def check_stmt(stmt: AssemblyStatement):
9     # will operate as expected
10    if isinstance(stmt, Assign):
11        # will always be False
12        if optional_stmt(stmt):
13            pass
```

Listing 6.9: Casting to Optional Type

6.4.3 Miscellaneous Changes

We now discuss other miscellaneous changes. Codon does not support `enum`, so all the `enums` converted into normal classes. Furthermore, Codon does not support `None` as a return type so it is instead replaced with `NoneType`. In addition, as mentioned in 6.5, `match` statement across types is not supported, so those had to be changed to chained `if/elif` statements.

6.5 Alternative Integrations with Codon

The current pipeline executes the code using `codon run`, which takes as input a JSON version of the IR via command-line arguments and outputs a JSON version of the instructions. However, Codon provides two features that allow easier integration with Python: (1) `pyext` and (2) Codon JIT. Surprisingly, code that compiles with `codon build` from the previous section does not work out-of-box with these two integration methods. This section explores which aspects could and could not compile for both of these methods.

6.5.1 PyExt

PyExt allows for the Codon code to automatically generate Python extensions. Python extensions are generally written in C, C++, or Cython and allow users to add built-in modules that can call C code and add new built-in Python types. [30] Codon can generate Python extensions when the Codon code is compiled with the `-pyext` flag:

```
1 codon build -pyext extension.codon
```

However, a fundamental limitation of Codon's PyExt feature is it does not allow for polymorphic types. This limitation essentially means that one type cannot satisfy multiple types. As shown in Listing 6.10, Codon's PyExt cannot support any inheritance and therefore does not allow for specific Statements and Expressions to be subtypes of an overarching Statement or Expression class.

```
1 @dataclass
2 class AssemblyNode:
3     pass
4
5 @dataclass
6 class AssemblyExpression(AssemblyNode): # does NOT compile with pyext
7     pass
```

Listing 6.10: Subclassing fails to compile with Codon Pyext

Not supporting polymorphic types is problematic since adding typing elements that are helpful for compiler development is one of the core tenants of this thesis! We nonetheless explored workarounds by creating one type for the entire pipeline that acts like a dictionary. We created a simple proof-of-concept with this method which can be expanded into the entire CFG conversion. As shown in 6.11, we additionally need to specify `python=True` to indicate that the dataclass can be used in Python.

```
1 @dataclass(python=True)
2 class AssemblyExpression():
3     _type: str
4     val: str
5
6     def __init__(self, _type: str, val: str = ""):
7         self._type = _type
8         self.val = val
```

Listing 6.11: Example AssemblyExpression

One limitation is that it does not support recursive types since it cannot automatically cast a user's input pyobj to the appropriate class, so we additionally created a simple function that can match across a field and create the correct class that the type checker can handle as shown in 6.12.

```
1 def create_expr(a: AssemblyExpression) -> AssemblyExpression:
2     if a._type == "int":
3         return AssemblyExpression(_type="int")
4     elif a._type == "str":
5         return AssemblyExpression(_type="str")
```

Listing 6.12: Creating an AssemblyExpression

Together, it is reasonable to create the full CFG conversion with Codon extension and is an area to further explore.

6.5.2 Codon JIT

Codon also supports Just-In-Time (JIT) compilation. It can compile both classes and functions itself.

To compile classes, Codon supports the `@codon.convert` decorator and requires a `--slots__` attribute for all the attributes in the class. The slots method could not recognize classes within classes, so to allow for compilation, dataclasses were no longer nested. An example of this syntax is shown in [6.13](#).

```
1 @codon.convert
2 @dataclass
3 class Assign(MyAssemblyStatement):
4     __slots__ = ("lhs", "rhs")
5
6     lhs: MyAssemblyExpression
7     rhs: MyAssemblyExpression
```

Listing 6.13: Codon JIT Example

To compile a function, one can simply add the `@codon.jit` decorator, and it will compile the code with Codon. This is shown in [6.14](#).

```
1 @codon.jit
2 def get_cond(stmt: AssemblyStatement) -> Optional[AssemblyExpression]:
3     if isinstance(stmt, If) or isinstance(stmt, IfElse) or isinstance(stmt,
4     , While):
5         return stmt.get_condition()
6     return None
```

Listing 6.14: Codon JIT Example for a Function

We were able to compile all the related functions and dataclasses with jit. The next step to make this approach fully compatible with Python is to find a way to convert an input to the appropriate classes. However, Codon does not have nice support for type casting of input Python objects at the moment. We can approach type casting in a similar manner as

our JSON parsing pipeline where we match strings to Codon types. However, this approach is similar to the original compilation method and still does not allow for a clean integration with Python.

Chapter 7

Evaluation

7.1 Evaluation Setup

To evaluate our pipeline, we implemented part of the Finch compiler [2]. More specifically, we implement Finch assembly instructions from finch-tensor-lite, a sparse tensor compiler written in Python.

7.1.1 Evaluation Metric

The primary metric we considered for our evaluation is the runtime to (1) build the CFG and (2) build the CFG and perform dataflow analyses across it. For our code, we consider the runtime to be converting our IR to the CFG and exclude the time to read the data in and out as JSON. With some of the potential extension we outlined in the previous sections, we hope to avoid this step in the future. For the Finch code, we consider the runtime to be the time converting the Finch assembly to the IR. We ran each operation three times and took the minimum runtime. We ran experiments on an Alienware m18 R2 with Intel® Core™ i9-14900HX running Ubuntu 24.04.4 LTS.

7.1.2 Validating Correctness

To evaluate that our implementation generates correct code, we created functions that convert the CFG back to flat instructions and converts that into Finch assembly. This can be executed by Finch’s interpreter to ensure it matches the results from the original Finch code.

Converting the CFG to a flat representation may be ambiguous. For example, a branch could indicate a if/else statement or a while loop. To remove this ambiguity, we added an annotation field to every instruction in the CFG which provided additional metadata of the corresponding expression or statement in our IR.

7.2 Results and Discussion

Table 7.1 show the runtime results. For the tests, we find the Codon is significantly faster than Finch or the Python implementation.

Function	Finch Runtime (ms)	Python Runtime (ms)	Codon Runtime (ms)
Dot Product	0.28	0.32	0.04

Table 7.1: Runtime of CFG Creation

7.2.1 Case Study: Dot Product

One test case was the dot product as used in Finch, shown in Figure 7.1.

```
1 def dot_product(a: np_buf_t(float64), b: np_buf_t(float64)) -> float64:
2     c: float64 = 0.0
3     a_: np_buf_t(float64) = unpack(a)
4     b_: np_buf_t(float64) = unpack(b)
5     for i in range(0, length(slot(a_, np_buf_t(float64)))):
6         c: float64 = add(c, mul(load(slot(a_, np_buf_t(float64))), i), load
7         (slot(b_, np_buf_t(float64)), i)))
8     repack(a_)
```

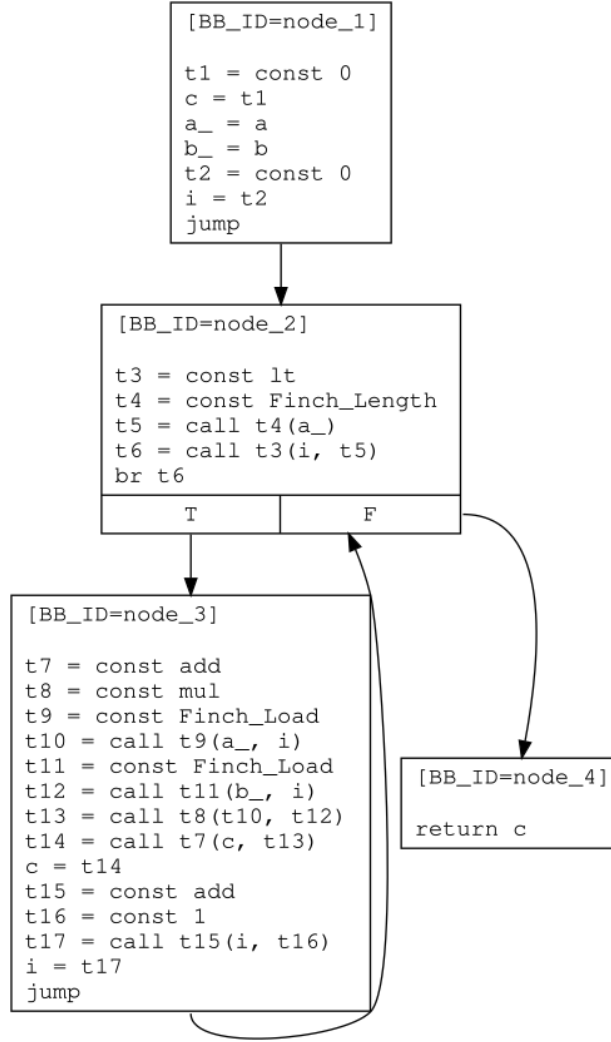



Figure 7.1: CFG generated from the dot product code

```

8   repack(b_)
9   return c

```

Listing 7.1: Finch Code for dot product

After going through the Codon and Python pipeline, it generates the CFG as shown in 7.1. The runtime recorded is the time it takes to compile from our IR to the CFG.

To evaluate that this CFG is indeed correct, we essentially flatten the CFG using metadata attached to each instruction. An example of the unwrapped code is shown in 7.2. In the example, the types are not reconstructed since they aren't necessary for validating that the code is correct but can be added from the metadata saved. Also note that there are

special Finch functions that are saved with the `Finch_` prefix. In the example, these are `Finch_Length` and `Finch_Load`.

```
1 def dot_product(a: object, b: object) -> object:
2     t1: object = 0
3     c: object = t1
4     a_: object = unpack(a)
5     b_: object = unpack(b)
6     t2: object = 0
7     i: object = t2
8     t3: object = lt
9     t4: object = Finch_Length
10    t5: object = t4(slot(a_, object))
11    t6: object = t3(i, t5)
12    while t6:
13        t7: object = add
14        t8: object = mul
15        t9: object = Finch_Load
16        t10: object = t9(slot(a_, object), i)
17        t11: object = Finch_Load
18        t12: object = t11(slot(b_, object), i)
19        t13: object = t8(t10, t12)
20        t14: object = t7(c, t13)
21        c: object = t14
22        t15: object = add
23        t16: object = 1
24        t17: object = t15(i, t16)
25        i: object = t17
26        t3: object = lt
27        t4: object = Finch_Length
28        t5: object = t4(slot(a_, object))
29        t6: object = t3(i, t5)
```

Listing 7.2: Recreated Finch Code for dot product

7.2.2 Discussion of Results

Codon gets a roughly 10x speed up with the compilation. This seems reasonable: most dataflow optimizations are generally fairly straightforward and recursive. Dataflow optimizations generally follow a match statement and recurse if appropriate. Parallelism, vectorization, tiling, and other low level optimizations are often not employed in dataflow algorithms. However, improvements with the recursion should be helpful or basic optimizations such as CSE. At the time of writing, Python does not support tail recursion elimination but C/C++ does, so these additional optimizations will likely help improve runtime.

Chapter 8

Future Work

8.1 Recommendations for Codon

When converting to Codon, properly converting the type system was one of the most difficult aspects. Many of the errors encountered are discussed in Chapter 6. A few non-intuitive aspects that could be reworked include losing information when casting type `T` to `Optional[T]`. In addition, casting Python objects (`pyobj`) proved to need extra mechanisms, especially for dataclasses that are recursive.

Another area which would be nice would be having similar behavior between `codon run` and `pyext` and/or `jit`. Currently for our code, converting between code that can be compiled with `codon run` to code that can be compiled with `pyext` or `jit` requires non-negligible rewrites. Alternatively, it would be helpful to document the limitations with the Python integrations.

Finally, it would be nice to have more support for manipulating `pyobjs` to a desired type in Codon since casting the `pyobj` requires hacky workarounds that only occasionally work. Alternatively, supporting recursive custom-defined types would remove the need to work with `pyobj` directly.

8.2 Extra Features

The thesis is a start to a Python framework for developing DSLs, and there are a handful of additional features that could be expanded on and/or reworked.

Firstly, in general, it would be nice to have more IRs and dataflow analyses. With more analyses implemented, it would be neat to explore how to exploit even more reuse between the analyses. In addition, with more IRs implemented, it would be nice to explore different abstractions when constructing the CFG. The current abstract for constructing the CFG works for the few languages we have defined so far, but there are certainly languages that fall outside the model, so there is likely room for more sophisticated abstraction models.

Finally, as mentioned in Chapter 6, there is much room for improving the Codon integration. Instead of passing JSON, it would be neat to directly input Python. In addition, we should make more aspects of the pipeline Codon-compatible. To start addressing this, we opened a few issues in the Codon repository.

Chapter 9

Conclusion

We present an exploration of implementing a flexible and comprehensive Python framework for developing DSLs. We provided an implementation of ADTs in Python which can be used to construct ASTs and other IRs like instructions for basic blocks. We next provided flexible functions that given functors that provide metadata on a language, can construct an SSA and non-SSA CFG and perform dataflow analyses. Finally, we convert part of the pipeline into Codon and show a notable speed up.

All code developed in this thesis is open-sourced at <https://github.com/PythonCompilers>.

References

- [1] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, “Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines,” in *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI ’13, Seattle, Washington, USA: Association for Computing Machinery, 2013, pp. 519–530, ISBN: 9781450320146. DOI: [10.1145/2491956.2462176](https://doi.org/10.1145/2491956.2462176). [Online]. Available: <https://doi.org/10.1145/2491956.2462176>.
- [2] W. Ahrens, T. F. Collin, R. Patel, K. Deeds, C. Hong, and S. Amarasinghe, “Finch: Sparse and structured tensor programming with control flow,” *Proc. ACM Program. Lang.*, vol. 9, no. OOPSLA1, Apr. 2025. DOI: [10.1145/3720473](https://doi.org/10.1145/3720473). [Online]. Available: <https://doi.org/10.1145/3720473>.
- [3] W. Thies, M. Karczmarek, and S. Amarasinghe, “Streamit: A language for streaming applications,” in *Compiler Construction*, ser. Lecture Notes in Computer Science, R. Horspool, Ed., vol. 2304, Berlin, Heidelberg: Springer, 2002, pp. 14–27. DOI: [10.1007/3-540-45937-5_14](https://doi.org/10.1007/3-540-45937-5_14).
- [4] Y. Zhang, M. Yang, R. Baghdadi, S. Kamil, J. Shun, and S. Amarasinghe, “Graphit: A high-performance graph dsl,” *Proc. ACM Program. Lang.*, vol. 2, no. OOPSLA, Oct. 2018. DOI: [10.1145/3276491](https://doi.org/10.1145/3276491). [Online]. Available: <https://doi.org/10.1145/3276491>.

- [5] F. Kjolstad, S. Kamil, S. Chou, D. Lugato, and S. Amarasinghe, “The tensor algebra compiler,” *Proc. ACM Program. Lang.*, vol. 1, no. OOPSLA, 77:1–77:29, Oct. 2017, ISSN: 2475-1421. DOI: [10.1145/3133901](https://doi.org/10.1145/3133901). [Online]. Available: <http://doi.acm.org/10.1145/3133901>.
- [6] C. J. Date and H. Darwen, *A guide to the SQL standard (4th ed.): a user’s guide to the standard database language SQL*. USA: Addison-Wesley Longman Publishing Co., Inc., 1997, ISBN: 0201964260.
- [7] D. Thomas and P. Moorby, *The Verilog Hardware Description Language*, 5th ed. Springer Publishing Company, Incorporated, 2008, ISBN: 0387849300.
- [8] D. J. Greaves, *Research note: An open source bluespec compiler*, 2019. arXiv: [1905.03746](https://arxiv.org/abs/1905.03746) [cs.PL]. [Online]. Available: <https://arxiv.org/abs/1905.03746>.
- [9] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, “MLIR: Scaling compiler infrastructure for domain specific computation,” in *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021, pp. 2–14. DOI: [10.1109/CGO51591.2021.9370308](https://doi.org/10.1109/CGO51591.2021.9370308).
- [10] T. Rompf, “Reflections on lms: Exploring front-end alternatives,” in *Proceedings of the 2016 7th ACM SIGPLAN Symposium on Scala*, ser. SCALA 2016, Amsterdam, Netherlands: Association for Computing Machinery, 2016, pp. 41–50, ISBN: 9781450346481. DOI: [10.1145/2998392.2998399](https://doi.org/10.1145/2998392.2998399). [Online]. Available: <https://doi.org/10.1145/2998392.2998399>.
- [11] A. Brahmakshatriya and S. Amarasinghe, “Buildit: A type-based multi-stage programming framework for code generation in c++,” in *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021, pp. 39–51. DOI: [10.1109/CGO51591.2021.9370333](https://doi.org/10.1109/CGO51591.2021.9370333).

- [12] J. Bezanson, S. Karpinski, V. B. Shah, and A. Edelman, *Julia: A fast dynamic language for technical computing*, 2012. arXiv: [1209.5145 \[cs.PL\]](https://arxiv.org/abs/1209.5145). [Online]. Available: <https://arxiv.org/abs/1209.5145>.
- [13] M. G. Tallada, J. Denny, P. Valero-Lara, S. Lee, K. Teranishi, and J. S. Vetter, “Ecc++ : A compiler construction framework for embedded domain-specific languages,” in *2024 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2024, pp. 667–677. DOI: [10.1109/IPDPSW63119.2024.00129](https://doi.org/10.1109/IPDPSW63119.2024.00129).
- [14] A. K. Sujeeth, K. J. Brown, H. Lee, T. Rompf, H. Chafi, M. Odersky, and K. Olukotun, “Delite: A compiler architecture for performance-oriented embedded domain-specific languages,” *ACM Trans. Embed. Comput. Syst.*, vol. 13, no. 4s, Apr. 2014, ISSN: 1539-9087. DOI: [10.1145/2584665](https://doi.org/10.1145/2584665). [Online]. Available: <https://doi.org/10.1145/2584665>.
- [15] J. Ansel, E. Yang, H. He, *et al.*, “Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS ’24, La Jolla, CA, USA: Association for Computing Machinery, 2024, pp. 929–947, ISBN: 9798400703850. DOI: [10.1145/3620665.3640366](https://doi.org/10.1145/3620665.3640366). [Online]. Available: <https://doi.org/10.1145/3620665.3640366>.
- [16] M. Willsey, C. Nandi, Y. R. Wang, O. Flatt, Z. Tatlock, and P. Panchekha, “Egg: Fast and extensible equality saturation,” *Proc. ACM Program. Lang.*, vol. 5, no. POPL, Jan. 2021. DOI: [10.1145/3434304](https://doi.org/10.1145/3434304). [Online]. Available: <https://doi.org/10.1145/3434304>.
- [17] S. Weirich, B. A. Yorgey, and T. Sheard, “Binders unbound,” in *Proceedings of the 16th ACM SIGPLAN International Conference on Functional Programming*,

- ser. ICFP '11, Tokyo, Japan: Association for Computing Machinery, 2011, pp. 333–345, ISBN: 9781450308656. DOI: [10.1145/2034773.2034818](https://doi.org/10.1145/2034773.2034818). [Online]. Available: <https://doi.org/10.1145/2034773.2034818>.
- [18] M. Fehr, M. Weber, C. Ulmann, A. Lopoukhine, M. Lücke, T. Degioanni, M. Steuwer, and T. Grosser, *Sidekick compilation with xdsl*, 2024. arXiv: [2311.07422](https://arxiv.org/abs/2311.07422) [cs.PL]. [Online]. Available: <https://arxiv.org/abs/2311.07422>.
- [19] M. Levental, A. Kamatar, R. Chard, K. Chard, and I. Foster, *Nelli: A lightweight frontend for mlir*, 2023. arXiv: [2307.16080](https://arxiv.org/abs/2307.16080) [cs.PL]. [Online]. Available: <https://arxiv.org/abs/2307.16080>.
- [20] A. Shajii, G. Ramirez, H. Smajlović, J. Ray, B. Berger, S. Amarasinghe, and I. Numanagić, “Codon: A compiler for high-performance pythonic applications and dsls,” in *Proceedings of the 32nd ACM SIGPLAN International Conference on Compiler Construction*, ser. CC 2023, Montréal, QC, Canada: Association for Computing Machinery, 2023, pp. 191–202, ISBN: 9798400700880. DOI: [10.1145/3578360.3580275](https://doi.org/10.1145/3578360.3580275). [Online]. Available: <https://doi.org/10.1145/3578360.3580275>.
- [21] P. developers, *Pyright*, Accessed: 2025-12-12, 2025. [Online]. Available: <https://github.com/microsoft/pyright>.
- [22] P. developers, *Pyre*, Accessed: 2025-12-12, 2025. [Online]. Available: <https://github.com/facebook/pyre-check>.
- [23] M. developers, *Mypy*, Accessed: 2025-12-12, 2025. [Online]. Available: <https://github.com/python/mypy>.
- [24] D. MacQueen, “Modules for standard ml,” in *Proceedings of the 1984 ACM Symposium on LISP and Functional Programming*, ser. LFP '84, Austin, Texas, USA: Association for Computing Machinery, 1984, pp. 198–207, ISBN: 0897911423. DOI:

- 10.1145/800055.802036. [Online]. Available:
<https://doi-org.libproxy.mit.edu/10.1145/800055.802036>.
- [25] L. of authors, *Static Single Assignment Book*. Springer, 2018.
- [26] S. K. Lam, A. Pitrou, and S. Seibert, “Numba: A llvm-based python jit compiler,” in *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, ser. LLVM ’15, Austin, Texas: Association for Computing Machinery, 2015, ISBN: 9781450340052. DOI: 10.1145/2833157.2833162. [Online]. Available: <https://doi.org/10.1145/2833157.2833162>.
- [27] A. Paszke, S. Gross, F. Massa, *et al.*, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems*, vol. 32, Curran Associates, Inc., 2019. [Online]. Available: <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html> (visited on 03/18/2024).
- [28] Y. Izawa, H. Masuhara, C. F. Bolz-Tereick, and Y. Cong, *Threaded code generation with a meta-tracing jit compiler*, 2022. arXiv: 2106.12496 [cs.PL]. [Online]. Available: <https://arxiv.org/abs/2106.12496>.
- [29] S. Behnel, R. Bradshaw, C. Citro, L. Dalcin, D. S. Seljebotn, and K. Smith, “Cython: The best of both worlds,” *Computing in Science Engineering*, vol. 13, no. 2, pp. 31–39, 2011. DOI: 10.1109/MCSE.2010.118.
- [30] *Extending python with c or c++*. [Online]. Available: <https://docs.python.org/3/extending/extending.html>.